FINAL DESIGN / F-1.0

오픈소스 AI 통제형 방화벽 최종 설계서

검증된 정책 변경 · 엄격한 수명 · 독립된 패킷 집행

Rust / nftables / MCP / 제한 위임 / 서명 증적

01 **하나의 구현 기준**

정책 의미와 실패 동작을 일관된 계약으로 확정

02 **선택의 근거까지 제공**

최신 원문 · 비교평가 · 실제 검증 범위를 함께 대조

03 **공개 개발을 위한 준비**

Apache-2.0 · 제3자 라이선스 · 기여와 보안 제보

2026.10.30 | 최종 설계와 통합 근거 · 라이선스 지침 수록

설계 단계: 실제 방화벽 제품 구현·실장비 검증 완료를 의미하지 않음

문서 안내

설계 기준판 F-1.0 | 2026년 10월 30일 | GitHub 공개 개발 기준

이 프로젝트는 Linux 서버·VM·미니 PC에서 실제 통신을 제어하고, 사람과 업무 에이전트의 접근 요청을 검증 가능한 정책으로 바꾸는 오픈소스 방화벽이다. AI는 선택적으로 요청 해석과 설명을 돕는다. 패킷의 허용 여부와 배포 권한은 결정적 코드, 승인된 범위, 서명된 실행 허가로 결정한다.

이 문서는 앞선 설계와 비교평가를 통합한 **최종 구현 기준**이다. 어느 모델의 설계를 그대로 정본으로 삼지 않는다. 기존 자료의 유용한 의도는 보존하되, 서로 다른 정책 의미를 동시에 구현하라고 요구하지 않는다. 설계 선택의 근거와 판본별 판단은 F_설계결정_및_통합근거.md에, 법률·배포 조건은 F_라이선스_및_GitHub_공개지침.md에 분리했다.

완료된 것은 설계와 참조 계약이다. 이 패키지는 실행 가능한 방화벽 제품, 실장비 호환 인증, 성능 실측 보고서가 아니다. JSON·문서·구성도 검증과 실제 커널·장비 시험을 구별한다. 여기의 수치는 별도 표기가 없으면 출시 시험의 목표값이다. 미구현 명령이나 존재하지 않는 저장소 다운로드 링크를 제공하지 않는다.

문서의 MUST는 해당 프로필 출시의 필수 조건이고, 후속 기능은 명시한 진입 시험을 통과하기 전까지 활성화하지 않는다. 본문·JSON 계약·시험 간 모순은 구현자가 임의로 선택할 문제가 아니라 출시를 중단하고 같은 변경에서 함께 수정할 결함이다. core-v1은 프로토콜·의미 규격 이름이며 제품 v1.0 출시 완료를 뜻하지 않는다.

읽는 순서

1~13장은 제품 범위·정책 의미·권한·실행과 복구, 14~20장은 AI·MCP·증적·확장·구현 구조, 21~25장은 검수·출시·공개 기준입니다. 부록 A는 양쪽 설계의 최신 판본을 대조한 통합 근거, 부록 B는 Apache-2.0과 GitHub 공개 지침입니다. 기계가 사용하는 JSON 계약과 참조검증기는 별도 F_개발계약 폴더에 있습니다.

목차

1. 최종 결정과 제품의 존재 이유 5

2. 첫 사용자, 지원 범위, 제공하지 않는 보장 6

3. 요구사항과 추적 가능한 완료 조건 7

4. 아키텍처와 권한 경계 8

5. 신뢰 모델과 반드시 유지할 불변조건 9

6. 정책 의미의 단일 정의 10

7. Linux 데이터 경로와 strict expiry 구현 계약 11

8. 자산, 서비스, workload identity 12

9. 접근 봉투, 위임, quota 12

10. 상태, 계획, 승인과 실행 허가 13

11. 결정적 검증과 증거 수준 14

12. 적용, fencing, UNKNOWN, rollback 15

13. 장애와 시각의 동작 계약 16

14. 내부 AI와 업무 에이전트 16

15. REST · MCP · CLI 계약 17

16. 데이터 모델, canonical hash, PBOM 19

17. 어댑터, 탐지와 장비 연동 20

18. 확장 기능의 진입 조건 21

19. 설치, UI, 운영 runbook 22

20. Rust 구현 구조와 개발 계약 23

21. 검증 계획과 성능·자원 목표 24

22. 출시 순서와 구현할 첫 작업 25

23. GitHub 공개, 기여와 Apache-2.0 26

24. 최종 인수 기준과 남겨 둔 경계 26

25. 출처, 입력 판본과 검증 범위 27

부록 A. 설계 결정과 통합 근거 28

부록 B. 라이선스와 GitHub 공개 37

1. 최종 결정과 제품의 존재 이유

핵심 가치는 자연어로 방화벽 명령을 생성하는 데 있지 않다. 누가 무엇을 왜 열었고, 허용 범위가 어떻게 검증됐으며, 장치에 실제 무엇이 남았는지를 같은 식별자와 증거로 연결하는 데 있다. AI를 끄더라도 이 가치와 실제 방화벽 기능은 유지된다.

영역	최종 결정	결정의 이유
기본 집행	Linux nftables L3/L4 필터	검증할 커널 의미를 작게 유지하면서 전용 장비 없이 사용
제품 코어	Rust, 모듈형 제어 서비스	도메인 타입·오류·자원 상한을 공유하고 초기 운영 복잡도 제한
화면	TypeScript strict + React 정적 번들	제어 서비스가 제공, 별도 Node 서버 불필요
정책 의미	기본 차단, deny 우선, 유효 allow의 합집합	사용자 priority·first-match 혼합으로 생기는 해석 차이 제거
정책 수명	allow 종료 시각과 deny 검토 시각 분리	차단 검토기한을 지나 자동으로 통신을 여는 오류 방지
세션	현재 허용 집합을 기존 연결에도 적용	이전 연결이라는 이유로 만료·차단을 우회하지 않음
자율 요청	접근 봉투 x 위임 x 신원 x quota	네트워크 범위와 요청자 권한을 서로 다른 계약으로 제한
승인	정확한 계획 해시에 사람 승인 또는 유효한 사전 위임	AI confidence·도구 실행 확인을 조직 승인으로 오인하지 않음
적용	단일 target 계획, 로컬 원자 적용, 관측 후 확정	DB·커널·여러 장비를 하나의 트랜잭션으로 가장하지 않음
장애	신규 권한 확대 중지, 기존 정책과 만료 유지	제어 장애와 데이터 경로 장애의 효과를 구분
증적	PBOM 해시 그래프와 서명 영수증	기록의 연결·변조 탐지를 제공하되 사실성과 인증을 과장하지 않음
확장	별도 프로세스 어댑터, 기능별 지원 등급	벤더·모델·탐지 시스템 전체를 한꺼번에 지원한다고 주장하지 않음
공개	자체 제작 코어·계약·문서에 Apache-2.0	특허 허여와 재사용 조건이 명시된 허용적 라이선스; 제3자 예외 별도

프로젝트의 공개 이름·도메인·상표는 별도 결정이다. 기존 작업명을 자동으로 사용권이 확인된 상표로 취급하지 않는다. 채용 과제의 회사명·로고·제출 정보는 공개 제품의 이름이나 보증 주체로 사용하지 않는다.

2. 첫 사용자, 지원 범위, 제공하지 않는 보장

첫 사용자는 Linux 서버 1~20대 또는 격리 개발망을 운영하는 개발자·소규모 인프라 팀이다. 첫 시연은 개발 워크로드 > 테스트 DB의 임시 접근을 요청하고, 검토·적용·반납·만료를 관찰하는 흐름으로 고정한다. 유료 AI·기업 CMDB·GitHub 계정·대형 SIEM은 선행 필수가 아니다.

프로필	core-v1 범위	선행 조건
Lab	3 network namespace 또는 3 VM, 단일 사용자·단일 조직	기존 업무망과 분리, 합성 자산·서비스
Host	Linux INPUT/OUTPUT, IPv4/IPv6	타 방화벽 관리자의 소유권 충돌 제거
Routed Gateway	두 데이터 인터페이스 사이 FORWARD	실제 경유 라우팅·대칭 반환 경로·관리 경로 확보
Team	PostgreSQL, 조직 인증, 독립 승인·서명	Lab 코어와 보안·복구 시험 후 별도 지원
Vendor import	읽기·가져오기·차분 보고서	적법한 장비 접근과 버전별 parser coverage
Vendor write	후속 기능별 검수 대상	의미 보존·배타 writer·회수·복구·실장비 시험

core-v1은 TCP/UDP 및 명시적 ICMP/ICMPv6 조건, 임시 allow, 지속 deny의 검토 수명, 상태 추적, 카운터와 제한 로그를 다룬다. 신규 임시 allow 최대치는 24시간이며 조직·위임의 더 작은 한도가 우선한다. 상시 업무 접근은 이 제한을 우회하는 영구 allow가 아니라 후속 service-lease 프로필의 별도 위험·갱신 설계로 다룬다.

NAT, FQDN의 동적 재해석, TLS 복호화, WAF, VPN, 투명 bridge, XDP, inline IPS, gateway HA는 core-v1에 포함하지 않는다. 기능을 폐기한 것이 아니라 독립된 의미와 실패 계약을 요구하는 확장으로 분리한 것이다. 인터넷 패키지 다운로드는 초기 데모에서 고정 자산으로 등록한 내부 패키지 미러·프록시를 이용한다. CDN IP를 넓게 허용해 도메인 보안이라고 표시하지 않는다.

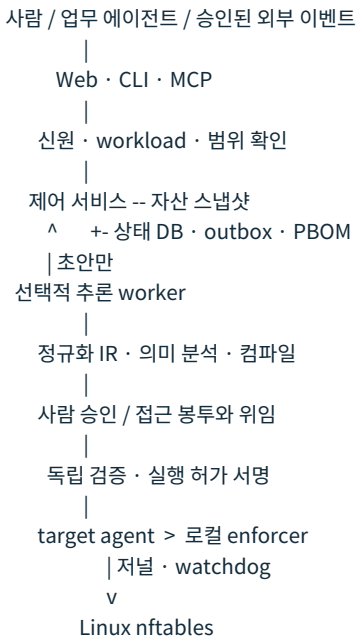
미니 PC를 설치하기만 하면 모든 통신이 보호되는 것은 아니다. 트래픽이 실제 해당 장치를 지나야 한다. 단일 gateway의 전원·OS·NIC 장애는 통신 단절을 일으킬 수 있다. Windows/macOS는 UI·CLI·개발 호스트로 사용할 수 있지만 집행 검수 대상은 Linux다. WSL2·컨테이너 실습 성공은 외부망 운영 게이트웨이 인증이 아니다.

3. 요구사항과 추적 가능한 완료 조건

ID	요구사항	구현 완료의 증거
R01	AI 없이 실제 필터링	수동 요청부터 TCP/UDP/IPv6 허용·차단·만료까지 실제 패킷 시험
R02	두 종류의 AI 분리	내부 추론 worker에 배포 권한 없음; 외부 agent에는 제한된 요청 API
R03	결정적 정책 의미	참조 평가기와 nft backend가 동일 fixture에서 같은 판정
R04	시각·세션까지 포함한 TTL	daemon 종료·controller 단절·기존 세션·재부팅 시험
R05	자율 개발 업무	봉투·위임 안에서 요청·승인 판정·dispatch·반납; 범위 초과 거절
R06	승인 내용과 실행 일치	계획·컴파일러·artifact·target 변경 시 재승인, replay 거절
R07	관측 가능한 적용	read-back 및 양성·음성·새 관리 연결 probe 후에만 ACTIVE
R08	안전한 실패	UNKNOWN 재조정, watchdog, 복구 실패의 DEGRADED 표시
R09	packet identity와 API identity 구별	독립 workload binding과 anti-spoof 경계 시험
R10	해석 가능한 감사	요청 > 정책 > 계획 > 승인 > 영수증 > 만료 PBOM 오프라인 검증
R11	독립 확장	어댑터 capability·coverage·적용 한계·maintainer 공개
R12	외부 기여 재현성	격리 lab, 계약 fixture, 개발 안내, 재현 가능한 CI
R13	공개 배포의 권리 관리	파일·의존성·이미지·모델·규칙별 라이선스와 출처 기록
R14	과장하지 않는 릴리스	설계 목표·실험 보고·재현 시험·운영 지원 상태를 분리

이 요구사항은 기능 목록과 시험 계획을 연결하는 기준이다. 기능이 문서에 있다는 이유로 지원됨으로 표기하지 않는다. 지원 상태는 designed, implemented, lab-verified, supported로 나누고 정확한 버전 조합을 기록한다.

4. 아키텍처와 권한 경계



실제 패킷: 송신 NIC > 커널 필터 > 목적지 NIC/서비스
AI·MCP·DB는 패킷별 처리 경로에 들어가지 않는다.

구성	소유 책임	허용하지 않는 권한
제어 서비스	요청, 자산 해석, 작업 상태, 검증 조정	임의 shell, 커널 쓰기, 배포키 직접 접근
추론 worker	원문 span 추출, 분류, 설명	승인·서명·장비 네트워크·비밀 저장소 접근
검증 worker	고정 입력의 의미 분석·컴파일·검사	장비 배포, 외부 비결정적 조회
release authority	승인·위임·artifact 재검사 후 grant 발급	자유 형식 명령 서명, DB boolean만 신뢰
target agent	outbound 통신, 저널, 관측·복구 조정	다른 target의 권한, 미서명 상태 변경
enforcer	지정 namespace·소유 nft table의 변경	전역 ruleset flush, 임의 파일·명령 실행
관측 connector	제한된 탐지·상태 이벤트 수집	경보를 곧바로 배포 승인으로 전환

모든 상자를 별도 서버로 배포하지 않는다. 하나의 controller에 요청·DB·MCP 모듈을 넣을 수 있다. 반면 추론 worker, 배포 signer, 특권 enforcer는 다른 UID·파일 권한·프로세스 경계를 유지한다. Lite도 이 경계를 없애지 않으며 여러 시스템 서비스로 한 장치에 설치할 수 있다. Lab의 단일 사용자 승인과 Team의 독립 승인 보장은 구분한다.

배포 서명키·호스트 root·커널·승인 관리자 전체가 동시에 침해된 환경은 본 소프트웨어만으로 방어할 수 없다. 동일 호스트의 UID 분리는 root 공격자에 대한 하드웨어 보안 경계가 아니다. Team은 signer와 승인 자격증명 저장소를 제어 서버와 분리할 수 있어야 한다.

5. 신뢰 모델과 반드시 유지할 불변조건

비신뢰 입력에는 자연어·로그·PR 설명·RAG 문서·외부 detector·모델 출력·사용자 제공 JSON·MCP client metadata가 포함된다. mTLS는 통신 상대의 키 소유를 확인하지만 payload 내용의 사실성을 자동으로 증명하지 않는다. 자산 registry도 시점·출처가 있는 관측과 관리 정보다.

불변조건	강제 위치
AI 출력은 승인·집행 명령이 될 수 없음	worker 출력 schema, 프로세스·네트워크 권한
필수 검증의 FAIL/UNKNOWN은 배포 불가	application service와 signer의 독립 검사
유효 allow 없는 업무 트래픽은 차단	reference evaluator와 kernel backend
deny는 겹치는 allow보다 우선	IR 의미·컴파일 순서·차분 시험
allow의 원래 만료 상한은 재시도·복구로 늘어나지 않음	artifact·grant·kernel TTL·restore
body의 org/principal을 인증 근거로 사용하지 않음	인증 문맥과 모든 자원 조회
검증·승인된 해시와 다른 실행은 불가	signer·agent·enforcer
한 enforcement scope에는 한 writer	소유권 등록·로컬 직렬화·fencing
네트워크 timeout은 성공도 미적용도 아님	UNKNOWN 상태와 read-back
rollback은 만료·철회된 허용을 부활시키지 않음	복구 계획·로컬 tombstone
조회 성공·queue ACK·HTTP 202는 ACTIVE가 아님	상태 machine와 UI·MCP 결과
겹치는 lease 중 하나 반납은 다른 정상 권한 삭제가 아님	허용 합집합·응답의 잔존 권한 표시
API의 신원만으로 패킷의 프로세스를 식별했다고 주장하지 않음	workload binding·설치 프로필
알려지지 않은 기능을 더 넓은 allow로 번역하지 않음	capability gate
감사 기록이 durable하지 않으면 새 적용을 시작하지 않음	fsync journal과 저장공간 예약
운영 정책·비밀·실제 증적은 공개 코드 저장소에 포함하지 않음	공개 준비와 CI secret scan

Prompt filter, 두 모델의 합의, typed AI output, temperature 0은 추가 품질 통제일 수 있으나 이 불변조건의 대체 수단이 아니다.

6. 정책 의미의 단일 정의

6.1 허용·차단 집합

시각 t에서 target의 업무 흐름 공간을 H라 하자. 활성 allow의 합집합을 A(t), 활성 deny의 합집합을 D(t)라 하면 업무 허용 집합은 P(t) = A(t) - D(t)다. 등록된 baseline의 좁은 관리·네트워크 제어 예외 B는 별도 보호 영역이며, 일반 업무 정책은 B를 수정할 수 없다. baseline과 충돌하는 격리 요청은 조용히 무시하지 않고 거절한다.

각 allow는 자신의 시작·종료 시간과 승인된 scope 안에서만 A에 기여한다. deny는 시작 이후 별도 승인된 삭제 전까지 D에 남는다. review_at은 경보·재검토 시각이며 삭제 시각이 아니다. core-v1에는 자동 만료되는 deny가 없다. deny 해제는 통신을 넓히는 새 계획으로 검증·승인한다.

사용자가 priority로 deny를 우회할 수 없다. 같은 의미의 규칙 순서를 바꿔도 P는 바뀌지 않아야 한다. 기존 벤더 first-match 설정을 가져올 때는 원래 순서를 보존한 별도 관측 모델로 분석한다. 순서를 버리고 union IR로 옮기지 않는다. 의미 동등성을 입증한 subset만 신규 관리 대상으로 전환한다.

6.2 흐름·방향·세션

정책의 direction은 **연결을 시작한 방향**이다. source/destination은 원래 요청의 방향이며, 응답 패킷은 독립적인 반대 방향 신규 허용을 의미하지 않는다. 평가 차원은 target·namespace·IP family·원본 source/destination·protocol·서비스 포트 또는 ICMP type/code·정/역방향 인터페이스·시간이다.

TCP와 UDP의 같은 숫자 포트는 다른 흐름이다. family나 protocol을 생략한 (src,dst,port) 키를 사용하지 않는다. 정책 scope는 필요 시 주소 범위를 표현하지만 실제 세션 식별에는 conntrack zone·원본 5-tuple·방향과 workload 격리 경계도 필요하다. source port를 정책 조건으로 사용하지 않는다는 사실이 세션 키에서 source port를 제거한다는 뜻은 아니다.

core-v1의 strict 의미는 **현재 P(t)에 포함되지 않는 후속 패킷을 통과시키지 않는 것**이다. allow A가 만료돼도 동일 흐름을 허용하는 B가 남아 있고 deny가 없다면 기존 세션은 계속 허용될 수 있다. A 반납 시 UI는 다른 유효 권한이 남아 통신이 계속될 수 있음을 보여준다. 개인 작업의 강제 세션 격리가 필요하면 공유 IP 권한이 아니라 별도 workload·네트워크 신원으로 분리해야 한다.

새 deny나 주소 축소는 기존 연결에도 즉시 적용되는 kernel transaction으로 반영한다. conntrack을 나중에 삭제하는 사용자 공간 작업에 보안을 의존하지 않는다. conntrack 정리는 자원 회수·RST 등 운영 보조 동작이며 정책 효력의 시작 조건이 아니다.

6.3 시간과 정규화

wire timestamp는 UTC YYYY-MM-DDTHH:MM:SSZ, 구간은 [not_before, expires_at)다. UI는 요청자의 시간대와 UTC를 함께 표시한다. 자연어 시각에 timezone이나 해석 근거가 없으면 NEEDS_INFO다. 정규화는 Unicode NFC, 엄격한 CIDR·IP parser, 중복 제거·집합 배열 정렬, port interval 병합을 포함한다.

host bits를 가진 CIDR, IPv4-mapped IPv6, family 불일치, 역전 포트, 시작보다 이른 종료를 묵시적으로 보정하지 않는다. 초안 수정으로 사용자에게 변경을 보여준 뒤 새로운 정책을 만든다. JSON 중복 키, NaN/Infinity, 알 수 없는 필드, 크기 상한 초과는 거절한다. 큰 generation은 JSON number가 아닌 정규 10진 문자열로 전달한다.

7. Linux 데이터 경로와 strict expiry 구현 계약

7.1 기본 경로

검수 대상은 NAT·flowtable·hardware offload·자동 conntrack helper가 없는 routed/host 프로필이다. 같은 패킷을 처리하는 전체 hook·priority·다른 manager·route를 조사한다. nft의 한 base chain에서 accept가 나왔다는 사실만으로 뒤의 다른 chain에서 drop되지 않는다고 가정하지 않는다. 컨테이너/CNI가 소유한 체인을 임의로 덮어쓰지 않는다.

논리 순서는 파싱·anti-spoof 및 필요한 커널 처리, 좁은 baseline, 업무 deny, 유효 allow의 새 흐름/응답 평가, 나머지 drop이다. spoof·비정상 source와 제어 프로토콜의 구체 조건은 설치 baseline에 서명해 고정한다. 루프백 전체 허용과 외부망 관리 예외를 같은 범위로 취급하지 않는다.

처음 구현하는 backend는 매 패킷에 현재 조건을 검사한다. established라는 이유만으로 전역 accept하는 최적화를 넣지 않는다. TCP/UDP 응답에는 conntrack original tuple과 ct direction을 사용하고, 돌아오는 인터페이스 쌍은 시작 방향의 반대 관계를 확인한다. conntrack NEW라는 표시만으로 중간 TCP ACK를 신규 연결로 수용하지 않도록 TCP 시작 조건과 loose tracking 설정을 시험한다.

7.2 커널 만료

allow revision마다 커널 timeout을 갖는 활성 토큰을 생성한다. 해당 정책의 새 흐름·기존 흐름을 허용하는 모든 branch는 같은 활성 토큰의 존재와 정확한 scope를 확인해야 한다. 여러 허용이 겹치면 각 branch가 독립적인 활성 토큰을 확인한다. 하나의 만료로 다른 branch의 권한을 삭제하지 않는다. token/map miss는 즉시 최종 drop이 아니라 다음 allow branch로 진행한다. 모든 유효 allow가 불일치한 뒤 기본 drop을 수행한다.

서명된 artifact에는 절대 시각을 저장한다. enforcer는 적용 직전 신뢰 시각의 상한(측정 UTC + 불확실성), 측정 이후 monotonic 경과, 적용 지연 예산을 제외하여 남은 TTL을 보수적으로 계산하고 초 단위로 내림한다. 이미 만료됐거나 안전 여유가 없으면 설치하지 않는다. 허용 branch에는 커널 상대 timeout과 절대 종료 시각의 패킷별 gate를 함께 요구한다. 절대 gate에도 시각 불확실성만큼 이른 cutoff를 적용해, 늦은 commit이 상대 timeout을 뒤로 밀어도 원래 종료를 넘겨 허용하지 않게 한다. 정확한 nft 표현과 시계 조건은 지원 kernel/tool 조합에서 실제 시험한다. nftables element timeout

이 보장은 검수한 시계 오차·최대 지연·재부팅/재개 제한 안에서 성립한다. 임의의 시계 조작과 무제한 OS 정지를 포함하는 실시간 보장을 주장하지 않는다. kernel lookup의 논리적 만료와 GC의 물리적 삭제 시점은 구분한다. 종료 직후 패킷 시험에는 commit 지연·벽시계 보정·중첩 lease를 포함하고 허용 오차를 결과에 공개한다.

미래 not_before의 자동 활성화는 커널 timeout만으로 해결되지 않는다. core-v1은 요청을 scheduled로 보관하고 시작 시각 이후 정상 검증 경로에서 snapshot·승인 유효성·위임·quota를 재확인해 새로운 짧은 grant를 발급한다. 내용이나 baseline이 바뀌면 새 계획·승인이 필요하다. offline 로컬 스케줄러가 오래된 grant로 새 권한을 활성화하지 않는다. controller 장애 때 예정 접근은 늦게 열리거나 열리지 않으며 일찍 열리지는 않는다. 독립적인 사전 승인 scheduled activation은 후속 계약이다.

7.3 mark와 ICMP

mark는 core-v1 정본의 필수 인가 근거가 아니다. 후속 성능 최적화에서만 (policy revision, epoch, scope)의 의미 보존 캐시로 도입한다. 예약된 비트 마스크·충돌·재사용 격리·만료·원자 invalidate를 증명해야 하고, 캐시 miss는 현재 허용 집합 재평가로 연결한다. TTL을 넘겨 살아 있는 mark만 보고 accept하지 않는다.

RELATED 전체 허용은 금지한다. IPv4 fragmentation-needed, IPv6 packet-too-big 등 오류 메시지는 검증된 부모 연결과 필요한 protocol 조건으로 제한한다. ND·RA·DHCP·routing 관련 제어 트래픽은 인터페이스별 baseline으로 분리한다. 외부에서 무조건 ICMPv6 전체를 허용하거나 차단하지 않는다. echo request/reply type 대응도 정책 의미와 컴파일 시험으로 고정한다.

fragment·IPv6 extension header·PMTU·비대칭 경로를 검사한다. 해석할 수 없는 패킷을 포트 0이나 ANY로 바꾸지 않는다. 허용 조건을 확인할 수 없는 패킷의 기본 결과는 drop이며, 필요한 제어 흐름은 별도 검증된 baseline으로 다룬다.

7.4 소유권과 적용

전용 inet table과 명명된 scope만 소유한다. flush ruleset은 금지한다. 고정 경로의 nft 실행 파일과 허용된 argument array·구조화 JSON 표준입력을 사용한다. shell 보간·사용자 nft 명령·임의 executable 경로는 받지 않는다. --check는 변경을 적용하지 않고 명령의 유효성을 검사하며 실망 reachability 보장이 아니다.

규칙·활성 토큰·정책 revision 전환을 하나의 nft transaction으로 수행하고 결과를 재조회한다. 원자성은 해당 커널 변경에 한정된다. DB·원격 장비·여러 호스트에는 자동 확장되지 않는다. nftables atomic replacement

8. 자산, 서비스, workload identity

자산의 안정된 ID는 이름과 IP보다 먼저 존재한다. 자산 revision은 주소·owner·org/project·zone·허용 인터페이스·출처·관측 시각을 포함한다. 외부 네트워크도 관리자가 등록한 자산으로 취급한다. 모델이 만든 IP·client가 주장한 프로젝트 이름은 신뢰된 자산을 생성하지 못한다.

resolver는 고정 snapshot에서 source/destination·service를 유일하게 해석한다. 0개 또는 복수 후보는 NEEDS_INFO다. 계획에는 snapshot hash와 capability hash를 넣는다. 기본 freshness 목표는 계획 시 5분, 배포 직전 target 관측 30초 이내다. 이를 보장하지 못하는 외부 CMDB는 수동 고정 snapshot 프로필로 구분하며, 자동화 수준을 낮춘다.

API 토큰은 요청자가 누구인지 알려줄 뿐 패킷을 보낸 프로세스를 식별하지 않는다. workload_binding은 신뢰된 launcher·orchestrator가 발급하며 org/project·실행환경·network namespace 또는 VM·source 주소·유효기간·attestation 출처를 결합한다. agent 자신이 binding과 자산 그룹을 수정하지 못한다.

공유 host IP나 NAT 출구 뒤 여러 worker가 있으면 IP 기반 허용은 같은 IP의 다른 프로세스에도 유효할 수 있다. 이 경우 host scope로 사실대로 표시한다. workload별 보안을 요구하면 별도 namespace/veth·VM/IP, 검수된 CNI identity, 또는 인증된 egress proxy를 사용한다. 단순 cgroup 숫자를 원격 gateway 패킷에서 알 수 있다고 가정하지 않는다. IP 재사용 전 기존 binding·lease를 회수하고, 재사용 유예와 anti-spoof 시험을 적용한다.

9. 접근 봉투, 위임, quota

AccessEnvelope는 사람이 사전 승인한 네트워크 최대 영역이고, Delegation은 어떤 신원이 어떤 도구로 그 영역 안의 권한을 요청할 수 있는지 정한다. 봉투가 있다는 이유만으로 모두 열지 않는다. 봉투는 lease가 활성화되기 전까지 실제 allow가 아니다.

자동 요청의 필요조건은 요청 범위 ⊆ 봉투 범위 ∩ 위임 범위 ∩ workload binding 범위 ∩ 조직 불변조건이다. service·target·자산·시간·환경·공개 노출·최대 TTL·quota를 모두 검사한다. 한도 초과 시 요청을 몰래 잘라 적용하지 않고 이유와 허용 한도를 반환한다. hard veto는 사람이 승인한 예외라는 문자열로 우회하지 못한다.

core-v1은 한 root delegation, 한 target 계획, 명시적 asset/service ID 집합을 지원한다. 계층 위임·동적 selector·사이트 간 quota는 후속 확장이다. 부모·자식 이름만으로 위임 사슬을 증명하거나 계층 수만 제한해 권한 확대를 막았다고 주장하지 않는다.

quota는 root·project 단위로 원자적으로 예약한다. 최소 항목은 활성 lease 수, 24시간 창의 신규 lease 수, 고유 목적지 수, 누적 승인 lease seconds, 동시 적용 수다. 만료 연장·여러 principal·작은 반복 요청으로 root 예산을 우회하지 못한다. 예산 계산의 시간 창·rounding·예약 수명은 버전 있는 조직 정책에 넣는다.

예약은 reserved > committed > released이며 UNKNOWN에서는 사용 중인 것으로 계산한다. grant 발급 직전 위임 철회·snapshot·quota를 다시 확인하고, 발급과 철회를 같은 직렬화 경계에서 처리한다. signer authority는 독립적으로 승인된 봉투·위임의 원본 bytes와 전용 권한 원장을 사용한다. controller는 authority의 quota·철회·승인 기록을 직접 갱신할 수 없고 예약 요청만 보낸다. 같은 PostgreSQL 서버를 사용해도 전용 DB principal/schema와 쓰기 권한을 분리한다. controller의 reservation 값은 요청 힌트이며 권한 근거가 아니다. 예산 확정·환급은 authority가 검증한 발급·회수·미적용 영수증과 일관된 상태에서만 수행한다.

연결이 끊긴 장치의 즉시 회수는 보장할 수 없다. 설치된 짧은 lease의 만료와 grant 시작 기한이 잔여 노출의 상한이며, 이 한계를 UI에 표시한다. authority 원장 장애는 자동 허용의 근거를 없애므로 새 grant를 중지한다.

위임 철회는 새 grant를 중단하고 연결된 target의 소유 lease 회수를 요청한다. 기존 allow를 만료보다 늦게 연장하지 않는다. 만료된 요청의 재시도는 새로운 승인 기회가 아니며 같은 idempotency key에는 기존 결과를 반환한다.

10. 상태, 계획, 승인과 실행 허가

정책 수명, 비동기 작업, target 적용 상태를 하나의 status에 섞지 않는다.

객체	주요 상태	금지되는 해석
요청	received, needs_info, rejected, planned, cancelled	요청 생성 성공을 통신 활성화로 해석
계획	ready, approval_required, approved, stale	내용 변경 후 이전 승인 재사용
operation	queued, running, succeeded, failed, cancelled, unknown	child target UNKNOWN인데 succeeded
target 적용	prepared, applying, verifying, active, unknown, rolling_back, rolled_back, degraded	API 200·candidate 저장만 보고 active
allow 수명	scheduled, active, expired, revoked	다른 작업 실패 때문에 종료 시각 연장
deny 수명	scheduled, active, review_due, removed	review_due를 자동 removed로 전환

Plan은 요청 revision, policy hashes, 자산 snapshot, target baseline·capability, compiler·validator digest, artifact, 검증 report, 새로 열린/닫힌 흐름, probe, 승인 조건, 복구 상한을 결합한 불변 객체다. core-v1 wire Plan은 한 target을 갖는다. 다중 target 변경은 여러 계획을 묶는 상위 workflow이며 상태를 target별로 보고한다.

승인에는 정확한 plan hash·승인자 신원·역할·scope·기한·철회 여부를 결합한다. 별도 ApprovalSet은 human 또는 delegated 중 한 경로를 명시한다. 불필요한 가짜 사람 승인을 생성하지 않는다. Lab은 소유자 1인 가능, Team production은 독립 network·service/security 검토자 2인, 고위험은 3인을 기본으로 한다. 자기 요청자는 독립 검토자 수에 포함하지 않는다.

가드레일 위반·미지원 의미·필수 UNKNOWN은 점수와 무관한 veto다. 위험 점수는 외부 노출, 범위, 서비스 민감도, target 수, 지속 시간, 세션 영향, 관측 불확실성의 우선순위 설명에만 쓴다. 승인 인원을 늘려 필수 검증 실패를 덮을 수 없다.

노출 차분 Δ+가 비어도 가용성 위험은 남는다. 전체 deny·필수 업무 중단·관리 경로 손상은 Δ-와 protected flow 검사를 별도로 거쳐 승인·중단한다. 미리 승인한 allow의 정상 만료·자기 lease 반납과, 임의의 대량 deny를 같은 축소 작업으로 취급하지 않는다.

signer는 DB의 approved=true만 신뢰하지 않는다. 검증된 artifact를 고정 입력으로 다시 확인하고, Git이면 정확한 review/commit/tree·role·철회 상태를 독립 credential로 확인한다. UI 승인은 plan hash에 결합한 승인키 또는 WebAuthn challenge를 검증한다. 위임이면 현재 delegation·봉투·quota·binding을 재확인한다. 로그인 성공과 내용에 대한 승인은 별개다.

DeploymentGrant에는 target, plan/artifact/baseline hash, 승인 증거 hash, requester, authority epoch, 증가 generation, 시작 허가 기한, confirm deadline, 제한된 복구 권한을 넣는다. 기본 grant 시작 유효기간은 5분, confirm은 60초다. 이것은 업무 lease의 TTL과 서로 다른 시간이다. 서명은 검증된 DSSE·Ed25519 라이브러리로 구현하며 암호 알고리즘을 직접 만들지 않는다.

11. 결정적 검증과 증거 수준

Gate	검사	실패 결과
G01	크기·JSON·schema·정규화	INVALID_INPUT
G02	인증·org/project·자원 소유권·workload binding	FORBIDDEN
G03	자산·service 해석과 snapshot freshness	NEEDS_INFO / STALE
G04	lifetime·관리 baseline·금지 범위·봉투·위임·quota	POLICY_DENIED
G05	capability·route·writer 소유권	UNSUPPORTED / CONFLICT
G06	현재/후보 의미, 시간 경계, 넓어지는/닫히는 흐름	FAIL / UNKNOWN
G07	compiler 결과·syntax·격리 커널 fixture·probe 계획	FAIL / UNKNOWN
G08	정확한 plan 승인 또는 위임 근거	APPROVAL_REQUIRED
G09	적용 직전 baseline·snapshot·grant·epoch 재검증	STALE / REPLAY
G10	실제 read-back·업무·음성·관리 probe	rollback / DEGRADED

형식 검증은 의미 검증이 아니다. CIDR 문자열 패턴·JSON Schema date-time의 통과만으로 주소·기간·권한이 유효한 것은 아니다. 참조검증기와 운영 검증기는 동일한 의미 시험을 공유하되 참조검증기 실행을 실제 커널 검증으로 표시하지 않는다.

core-v1의 집합 분석은 IP/port interval·family·protocol·인터페이스·시간 경계를 분할한 모델로 시작한다. BDD는 구현 선택이며 필수 제품 의존성이 아니다. 분석 node/partition·메모리·wall time 상한을 동시에 둔다. 예산 초과, parser 미지원, 필수 토폴로지 누락은 UNKNOWN이다. 필수 UNKNOWN은 signer까지 전달되어 적용을 차단한다.

분석해야 할 차분은 P(candidate,t)-P(baseline,t)와 그 역차분이다. 시작·종료·검토 시각을 구분하고, 중첩 lease의 서로 다른 만료 경계를 모두 평가한다. 표본 probe 몇 개의 성공을 전체 동등성 증명으로 표시하지 않는다. 검증 report에는 모델의 포함·제외 차원과 counterexample를 남긴다.

OPA는 조직 규칙 확장, Batfish는 지원하는 네트워크 구성의 추가 분석에 사용한다. nft parser가 없는 도구에 nft 안전성을 증명했다고 표시하지 않는다. OPA bundle은 입력 시각·revision을 고정하고 네트워크·현재 시각·난수에 의존하는 정책을 배포 판단에서 금지한다. native 검증기의 필수 안전 검사를 OPA가 느슨하게 덮어쓸 수 없다.

12. 적용, fencing, UNKNOWN, rollback

12.1 단일 target 적용

1. target의 단일 writer lease를 확보하고 영속 epoch/generation을 할당한다.
2. agent와 enforcer가 서명·대상·plan·baseline·승인·시각·generation을 각각 검사한다.
3. 현재 상태를 재조회한다. 차이가 있으면 STALE로 종료하며 자동 덮어쓰지 않는다.
4. old/candidate artifact, grant, tombstone, 복구 deadline을 로컬 journal에 fsync한다.
5. controller와 독립된 로컬 watchdog을 먼저 가동한다. 커널 transaction을 적용한다.
6. semantic read-back, 허용/차단 probe, 별도 관측자의 새 관리 연결 probe를 실행한다.
7. 동일 plan/generation의 서명된 confirm을 durable하게 저장한 뒤 ACTIVE로 확정한다.
8. 기한 내 확인되지 않으면 허가된 복구 절차를 수행한다. 복구 검증 실패는 DEGRADED다.

highest accepted generation과 journal은 재시작 후에도 유지한다. 같은 grant·같은 내용의 재전달에는 저장한 결과를 반환한다. 같은 generation의 다른 내용, 이전 epoch·generation, 다른 target의 grant는 거절한다. DB restore가 target의 generation을 되돌릴 수 없다.

watchdog도 operation ID와 자신이 설치한 generation·candidate hash에 결합한다. confirm과 timeout 복구는 같은 로컬 직렬화 경계에서 CAS로 경쟁하며 하나만 이긴다. 현재 generation/hash가 달라지면 오래된 watchdog은 새 상태를 되돌리지 못한다. 복구는 집행 generation을 감소시키지 않으며 최고 accepted generation을 보존한다.

kernel transaction과 DB 기록 사이에는 crash window가 있다. 복구 시 journal과 actual read-back으로 old/new/unknown을 판정한다. 네트워크 timeout 직후 무조건 apply를 재시도하지 않는다. UNKNOWN target은 잠금·quota를 유지하며 후속 충돌 변경을 중지한다. 관측으로 안전한 상태를 판정할 수 없으면 OOB 복구를 요구한다.

12.2 안전한 복구의 의미

이전 raw ruleset을 그대로 재설치하지 않는다. 이전 정책에서 이미 만료·철회된 allow를 제거하고, tombstone 및 적용 이후 더 강한 유효 deny를 존중하는 복구 후보를 만든다. 새 deny를 제거하는 복구가 필요하면 그 제거 영향까지 미리 승인한 rollback 범위가 있어야 한다. 복구가 허가된 흐름 상한을 만족하지 못하면 자동 복구를 하지 않고 좁은 baseline과 DEGRADED 상태로 남는다.

가용성 복구와 보안 범위 보존이 충돌할 수 있으므로 Plan에 선택한 rollback profile과 영향 상한을 명시한다. 중단된 TCP 세션·이미 전달된 데이터·외부 시스템의 부작용까지 되돌린다고 보장하지 않는다. 전체 conntrack flush는 일반 복구 수단으로 금지한다.

12.3 다중 대상과 외부 장비

후속 workflow는 작은 canary wave, target별 상태, 중지 기준, 보상 복구를 갖는다. 일부 성공은 PARTIAL이며 전체 원자 성공이 아니다. 연쇄 경로의 순서와 중간 노출을 계획에 포함한다. core-v1 schema 하나에 여러 target을 억지로 넣어 해시·승인 의미를 바꾸지 않는다.

벤더의 generation anchor를 읽고 나중에 쓰는 것만으로 stale writer를 막을 수 없다. 네트워크 단절 후 늦게 도착한 writer를 장비가 거절하려면 native CAS/lock/fencing 또는 그와 동등하게 검수된 배타 경로가 필요하다. 이런 보장이 없는 adapter는 exclusive-single-writer 수준으로 제한하고 자동 HA writer failover를 제공하지 않는다. 단순 선조회 비교를 linearizable CAS로 표기하지 않는다.

13. 장애와 시각의 동작 계약

장애	데이터 경로	신규 변경·복구
AI·MCP 장애	기존 kernel 정책 유지	수동 경로 가능; AI 필수화 금지
controller·DB 장애	기존 정책과 kernel TTL 유지	신규 grant 중단, 재연결 후 reconcile
agent 종료	kernel 필터·timeout 유지	systemd 재시작, journal 확인
enforcer 적용 중 종료	old/new/unknown 확인 필요	watchdog과 read-back으로 복구
감사 디스크 포화	현재 집행 유지	durable journal 불가 시 새 적용 금지
시각 신뢰 상실	기존 커널 상대 TTL은 연장하지 않음	신규 임시 allow·미래 활성화 중지
OS 재부팅	먼저 좁은 baseline 복원	신뢰 시각·원장 확인 전 임시 allow 복원 금지
target 전원·NIC 장애	gateway 통신 단절 가능	별도 검수된 HA 없으면 자동 우회 보장 없음
외부 API timeout	실제 결과 불명확	UNKNOWN, read-back 이전 반복 쓰기 금지
rollback 실패	보안·가용성 미확정	wave 중단, OOB 복구, 사고 증적

시간 보안은 단순 NTP 설정 여부로 끝나지 않는다. 적용 당시 trusted UTC와 monotonic 기준을 기록하고, 이후 벽시계 역행으로 TTL을 연장하지 않는다. 기본 허용 skew 목표는 2초이며 정확한 기준은 프로필에 서명한다. 시각 동기화가 불안정한 target은 새 권한을 받지 못한다.

운영 gateway의 suspend/hibernate는 지원 프로필에서 비활성화한다. 재개를 지원하려면 timer가 suspend 시간을 반영하는지 시험하고, 신뢰 시각 재확인 전 임시 권한을 격리해야 한다. VM snapshot restore도 일반 재시작과 다르며 오래된 키·generation·lease 재생 공격에 대한 격리와 재등록을 요구한다.

원격 controller가 끊기면 정책을 전부 비우는 식의 fail-open을 하지 않는다. 동시에 controller 장애를 이유로 만료 예정 allow를 무기한 유지하지도 않는다. fail-closed는 적용 경로·업무 경로·관리 복구 경로 각각 무엇을 닫는지 구체적으로 문서화한다.

14. 내부 AI와 업무 에이전트

내부 추론 AI는 원문 span·추출 후보·missing fields·근거 ID·설명을 생성한다. 자산을 확정하거나 승인·배포하지 않는다. span은 정규화 이전 원문 hash와 UTF-8 byte offset의 반열린 구간으로 정의하고, 원문에서 실제 동일 문자열이 나오는지 확인한다. span 일치가 해석의 정확성까지 증명하지는 않는다.

AI 결과는 draft, needs_info, abstain, failed로 구별한다. 없는 정보를 임의 기본값으로 채우지 않는다. 제공한 context manifest에 없는 evidence ID, 승인 필드, shell·장비 명령, 미등록 endpoint는 거절한다. prompt injection 방어는 입력 필터와 별개로 worker의 권한·egress·도구 범위를 제한하는 구조로 강제한다.

외부 업무 에이전트는 등록된 principal로 요청 API를 사용한다. Codex·Claude Code·Hermes·Antigravity·Grok 등은 가능한 client 종류이며, 이름이나 User-Agent는 권한 증거가 아니다. 실제 호환성은 client 버전·transport·인증·도구·비동기 응답 시험으로 표시한다. 여러 agent의 동의는 조직의 독립 사람 승인으로 계산하지 않는다.

AI provider는 생성형 API, 로컬 추론 HTTP, typed decision, ONNX inference, 공식 사용자 로그인 bridge를 별도 adapter로 분류한다. local 모델의 예외적인 cloud fallback은 기본 금지한다. restricted 데이터는 local-only이며 운영자가 명시적으로 바꾼 데

이터 정책 없이는 외부로 보내지 않는다. 구독 로그인, API 과금, 제품 관리자 로그인은 다른 권한이다.

Laya와 TypeSafe Jev는 식별된 typed decision 후보로 보존한다. Laya는 공개 저장소가 확인됐고, TypeSafe는 state와 typed question에 대한 구조화된 결과를 문서화한다. 둘의 실제 serving revision·질문 타입·confidence 필드·한국어 성능은 각각 검수하며 서로 완전 호환이라고 가정하지 않는다. 공급자의 속도·정확도 수치를 이 방화벽의 실측으로 인용하지 않는다. Laya 공식 저장소, TypeSafe 소개

confidence는 공급자가 정의한 통계일 뿐 승인 확률이나 안전성 증명이 아니다. 기본 체크포인트를 바로 가드·승인 판단에 승격하지 않고 도메인 평가·필요한 특화·보정을 거친다. 공식 로그인·SDK·preview 경로는 출시 시 공식 문서와 설치 버전을 검증하고 지원 조건에 맞는 adapter에서만 사용한다. 개인 구독 credential을 공용 서버에서 몰래 재사용하지 않는다.

운영 일꾼도 보존한다. drift 조사, 불필요 정책 후보, 사건 요약, 감사 자료 정리는 요청자와 다른 작업 역할이다. 결과는 정형 제안이며 직접 배포하지 않는다. 후속 worker queue의 claim에는 claim_generation과 불투명 token을 넣고 heartbeat·complete·파생 제안 제출을 현재 소유자와 CAS한다. 만료된 claim의 늦은 결과는 거절한다. 재시도 idempotency key는 작업 ID와 action revision에 결합한다. core-v1 필수 배포 구성은 아니지만 작업 프레임워크를 도입할 때 이 경계를 생략할 수 없다.

모델 평가에는 한국어·영어, 모호한 자산, 시간대, 다중 서비스, 악성 문장, 근거 부족을 포함한다. 필드 정확도·자산 환각·기권·사람 수정량·latency·비용·calibration을 측정한다. 모델 오답률과 안전 gate가 막은 비율을 분리한다. 새 모델은 digest/revision과 회귀 dataset을 고정하고 shadow 평가 후 승격한다.

15. REST·MCP·CLI 계약

Web, REST, CLI, MCP는 같은 application service를 호출한다. transport마다 별도의 느슨한 검증 경로를 만들지 않는다. 일반 API는 /api/v1, UTF-8 JSON, org/project를 인증 문맥에서 결정한다. mutation에는 idempotency key, draft 수정에는 expected revision/If-Match, 조회에는 제한 pagination을 둔다.

REST 경로	권한과 처리
GET /capabilities	인증 scope 내 기능과 제한 조회
POST /access-requests	자산·service·binding을 가진 제한 요청, 202 operation
GET /plans/{id}	자신의 불변 계획과 별도 승인 상태 조회
POST /plans/{id}/approvals	사람 reviewer 전용, hash에 결합한 승인 증거
POST /deployments	plan ID/hash·승인 근거 확인 후 202 operation
GET /operations/{id}	자신의 작업 종류·단계·실제 target 결과
POST /leases/{id}/release	자기 allow lease 회수 요청, 202 operation
GET /evidence/{id}	scope·민감정보 필터를 거친 증적 조회

이 경로는 구현할 API 계약이며 이미 실행 중인 endpoint가 아니다. 안정 오류 코드는 INVALID_INPUT, FORBIDDEN, NEEDS_INFO, POLICY_DENIED, APPROVAL_REQUIRED, STALE, UNSUPPORTED, QUOTA_EXCEEDED, UNKNOWN_COMMIT, VERIFICATION_FAILED, RESULT_TOO_LARGE다. HTTP는 인증 401/권한 403, 민감한 교차 조직 자원 404, revision·멱등 충돌 409, 의미 오류 422, 누락된 조건부 수정 428, 자원 한도 429, 의존 장애 503으로 구분한다. 내부 secret·전체 장비 설정·stack trace를 오류에 포함하지 않는다.

core-v1 MCP는 다음 여섯 동작으로 한 바퀴가 완성된다. 상세 기계 계약은 F_개발계약/을 사용한다.

도구	입력의 의미	서버 동작
get_capabilities	빈 객체	현재 principal의 기능·버전·한도 조회
request_access	자산·service·binding·기간·업무 근거	요청 생성, 해석·검증·계획·위임 판단; 아직 적용하지 않음
get_plan	plan ID	불변 계획·해시·영향·검증·승인 상태 조회
request_deployment	정확한 plan ID/hash·idempotency key	유효 승인 확인 후 제한된 dispatch 요청
get_operation	operation ID	진행·target 실제 결과·lease 상태 조회
release_access	자기 lease ID·사유·idempotency key	소유 allow 회수, 다른 권한 잔존 여부 표시

수동 사람 승인은 UI/별도 승인 API가 수행하고 MCP에 approve_any를 만들지 않는다. 범위 안의 자동 요청은 backend가 delegated ApprovalSet을 생성한다. renewal은 기존 lease를 덮어쓰는 연장이 아니라 현재 권한을 다시 평가한 새 request다. 원시 nft·shell·장비 API proxy·trust 수정·role 수정 도구는 없다.

도구별 성공 상태와 payload를 tagged union으로 묶는다. 예를 들어 get_plan 성공에는 실제 plan과 hash가 있어야 하며, active + data:null은 schema 단계에서 거절한다. succeeded operation에 UNKNOWN target이 들어갈 수 없다. accepted는 작업 접수일 뿐 active가 아니다. operation의 succeeded는 해당 동작의 완료다. 검증 완료는 plan 준비, 배포 완료는 target ACTIVE, 반납 완료는 해당 lease REVOKED로 매핑하고 kind와 결과를 함께 확인한다.

core-v1의 계획 조회는 제한된 완전 객체를 반환한다. 최대 정책 256건과 응답 256 KiB를 모두 만족해야 하며 초과 시 RESULT_TOO_LARGE로 거절한다. MCP cursor/pagination은 후속 확장으로 명시하고 입력에 없는 cursor를 지원한다고 광고하지 않는다. REST 목록의 pagination과 구별한다. 안전 판단용 내부 입력을 출력 제한 때문에 잘라 검사하지 않는다.

MCP 규격과 구현 라이브러리 버전은 각각 고정한다. 확인된 2026-07-28 규격의 요청별 metadata/discovery와 이전 2025-11-25 initialize 흐름을 별도 호환 경로로 다룬다. 실제 SDK의 지원과 client 계약 시험을 통과한 버전만 광고한다. 협상 실패를 무조건 구버전으로 강등하지 않는다. MCP 규격 변경

stdio는 stdout에 protocol만 쓰고 stderr에 제한 진단을 남긴다. OS 보호 launcher가 project credential을 제공하며 전체 사용자 환경을 복제하지 않는다. remote HTTP는 TLS, audience/resource·issuer 검증, 필요한 OAuth metadata와 scope를 사용한다. session ID·request ID·도구 annotation은 인증 수단이 아니다. 들어온 토큰을 장비·GitHub·모델 API에 전달하지 않는다.

입력 기본 64 KiB, 출력 256 KiB, tool 초기 응답 목표 2초, 긴 작업은 operation 조회로 처리한다. polling은 기본 5초에서 최대 60초까지 backoff한다. transport 취소·client 종료가 이미 적용한 lease 삭제를 뜻하지 않는다. lease 반납과 kernel 만료는 독립적인 수명 동작이다.

16. 데이터 모델, canonical hash, PBOM

저장 대상	책임	일관성 규칙
자산·service·binding revisions	해석과 소유권 근거	이름 변경으로 과거 snapshot 덮어쓰기 금지
requests·plans·policies	요청과 desired 의미	plan·policy revision 불변
envelopes·delegations·approvals	권한 근거	변경·철회 이벤트와 서명 검증
quota reservations	자율 작업 예산	root/project 단위 직렬화
operations·deployment targets	진행 상태	외부 부작용은 별도 read-back
receipts·PBOM events	관측·승인·수명 증거	append-only + 해시 참조
outbox·inbox dedup	재시도·중복 처리	상태 변경과 outbox 동일 DB transaction
secret references	credential 위치	비밀 실물·refresh token을 PBOM에 넣지 않음

Lite는 단일 조직의 SQLite WAL, 로컬 디스크만 사용한다. 여러 controller가 공유 파일을 쓰지 않는다. Team은 PostgreSQL·행 잠금·RLS를 사용하되 관리자·migration 우회 권한을 별도로 관리한다. 초기 큐는 DB outbox이며 Kafka/NATS는 측정된 병목이 생길 때만 도입한다.

정본의 책임도 분리한다. Git은 검토된 의도와 승인 이력, controller DB는 작업 상태와 desired revision, authority 원장은 유효 승인·위임·예산, target은 실제 observed 집행 상태, PBOM은 이들의 시점별 증거를 소유한다. Git commit 하나나 DB ACTIVE 값이 실제 장비 상태를 대신하지 않는다. 복구·drift는 원본을 임의 우선순위로 덮는 것이 아니라 이 책임에 따라 새 관측·계획으로 조정한다.

canonical 의미 객체와 runtime wire는 구별한다. plan_hash = SHA-256(JCS(PlanContent))이며 자신의 hash·승인·서명은 PlanContent에 넣지 않는다. 승인은 plan hash를, grant는 승인 증거와 artifact를, 영수증은 grant와 observed 상태를, PBOM manifest는 이 객체들을 참조한다. 순환 해시를 만들지 않는다.

JCS 이전 도메인 정규화 버전과 순서 의미가 없는 배열의 정렬 규칙을 명시한다. signature envelope와 그 payload, portable artifact와 적용 시점의 TTL을 포함한 nft JSON은 서로 다른 해시다. read-back semantic fingerprint에서 counter·조회 시각·감소 중인 remaining timeout을 제외하되, 실제 종료 상한·policy revision·scope는 비교한다.

JCS 자체는 Unicode NFC나 배열 정렬을 수행하지 않는다. 외부 issuer·subject·원문 증거·이미 서명된 payload에는 일괄 NFC를 적용하지 않고 해당 외부 계약의 원래 bytes를 보존한다. DSSE는 payloadType과 직렬화된 payload에 대한 PAE를 서명한다. 검증한 동일 payload bytes를 파싱해 사용하고 별도의 재직렬화 결과로 대체하지 않는다. keyid는 키 탐색 힌트이며 신뢰 root·scope 검증 없이 인가 근거가 될 수 없다.

PBOM에는 원문 hash, requester/initiator/workload, 모델·context·결과, snapshot, 검증 범위·반례, 승인·위임·quota, grant, target 결과, 실제 회수·복구를 연결한다. 원문 payload·개인정보는 기본 수집하지 않고 분류·보존 정책에 따라 별도 저장한다. 초기 보존 예시는 AI 원문 7일, 비식별 사용량 30일, 정책 증적 365일이며 조직 책임자가 조정한다.

append-only DB와 hash chain만으로 관리자 조작까지 방지했다고 주장하지 않는다. Team은 독립 저장소에 주기적으로 서명 checkpoint를 보관한다. 공개 transparency log에 실제 내부 IP나 운영 정책을 올리지 않는다. 오프라인 PBOM 검증은 참조·해시·서명의 무결성을 확인하며 장비가 과거 실제로 안전했다는 사실을 새로 증명하지 않는다.

17. 어댑터, 탐지와 장비 연동

어댑터는 순수 변환과 실제 IO를 나눈다. PureAdapter는 parse/import/render/semantic diff를 fixture로 시험하고, TargetAdapter는 capability/read/prepare/apply/observe/confirm/rollback을 담당한다. controller 안에 임의 동적 라이브러리를 로드하지 않고 별도 프로세스의 버전 있는 RPC를 사용한다.

capability는 제품명보다 구체적이어야 한다. OS/kernel/firmware·API 버전, IR subset, family·direction·protocol, native expiry·strict session, 원자성 범위, read-back fidelity, lock/fencing, rollback, 최대 크기, 알려진 gap을 포함한다. 쓰기 지원은 이 조합별 시험을 통과했을 때만 활성화한다.

연동	역할·우선순위	핵심 제한
nftables	필수 native 집행	소유 table·strict TTL·원자 전환 검수
NetBox	선택적 자산 source	snapshot provenance·freshness·수동 자산과 충돌 처리
OPA	조직 가드레일	고정 bundle, 기본 안전 조건 완화 금지
Batfish	지원 topology 분석	parser 경고·모델 coverage 공개
Suricata EVE·Zeek	탐지·프로토콜 관측	IDS 이벤트와 inline IPS 분리
Sigma/pySigma	로그 탐지 규칙 변환	firewall IR로 직접 번역 금지
FortiGate/FortiManager	read/diff 후 검수된 쓰기	VDOM/ADOM·package writer 단일화·task/read-back
PAN-OS/Panorama	read/diff 후 검수된 쓰기	candidate와 running 구별·남의 candidate 동반 commit 금지
QRadar/SOAR	사건·탐지·workflow	제품명을 패킷 집행기로 간주하지 않음
Cilium/Kubernetes	후속 CNI adapter	selector/identity·deny 의미·CNI 소유권 별도
AWX/OpenTofu 등	선택적 조직 실행기	direct writer와 같은 scope에 중복 배치 금지

탐지 이벤트는 출처·rule digest·event/ingest time·attribution·evidence를 가진다. 공유 NAT·proxy IP를 공격 주체로 단정하지 않는다. 자동 대응은 첫 버전 기본 off다. 후속 대응 template은 대상·최대 영향·rate·짧은 유효 권한·해제 승인·critical asset 제외를 별도로 검증한다. LLM이 자신의 자동 대응 범위를 늘릴 수 없다.

18. 확장 기능의 진입 조건

확장	보존할 제품 가치	개발 착수 전/출시 전 닫아야 할 계약
NAT·인터넷 edge	미니 PC의 실제 인터넷 gateway 사용	변환 전후 tuple, hook 순서, 세션 회수, 포트 충돌, IPv6, rollback
FQDN·패키지 egress	agent의 저장소·CDN 접근	DNS provenance/TTL·rebinding·CNAME·공유 IP·허용 하한/상한·만료
XDP	검증된 고 PPS prefilter	nft 의미와 이중 writer 방지, map generation, attach 실패, GPL helper 조건
Suricata inline IPS	서명 기반 패킷 검사	NFQUEUE/AF_PACKET 경로, 양방향, queue overflow, bypass, 장애·부하
gateway HA	단일 장치 장애 완화	split-brain·fencing·policy/lease generation·conntrack 동기화·오프라인 만료
bridge	투명 배치	L2/ARP/ND/VLAN·bridge hook·우회·재부팅 시험
계층 위임	조직 > orchestrator > worker 자율 업무	scope 축소 증명·부모 철회·root 합산 quota·분할 동작
다지역	site별 독립 운영	권한 partition·site lease·quota 분할·재연결 조정
운영 일꾼	조사·정리·감사 보조	claim generation·CAS·결과 schema·제안만 허용·중복 작업 통제
WAF/VPN/L7	application/tunnel 보안	별도 schema·키 수명·TLS·개인정보·route 의미

FQDN 근사는 allow와 deny에서 같은 방향으로 안전하지 않다. 허용 집합을 과대근사하면 과도한 노출을 탐지하는 데 도움이 될 수 있지만, deny까지 과대근사하면 실제로는 열린 흐름을 닫혔다고 오판할 수 있다. 후속 분석은 허용 상·하한을 별도로 계산하거나 불확실한 차원을 UNKNOWN으로 남긴다. core-v1은 DNS 이름을 고정 CIDR와 동등한 식별자로 지원하지 않는다.

XDP는 nft보다 앞에서 패킷을 drop할 수 있지만 XDP_PASS는 최종 허용이 아니다. inline IPS의 fail-open은 기본 firewall까지 무조건 열라는 뜻이 아니다. NFQUEUE의 listener 부재 bypass와 queue-full 동작은 별도 설정·시험 대상이다. 후자는 NFQA_CFG_F_FAIL_OPEN 등 실제 설정을 확인한다. Netfilter queue API VRRP 주소 전환만으로 세션·정책·lease가 동기화된 HA가 되지 않는다. 확장의 이름이 아니라 실패 시 실제 패킷 결과로 지원을 판단한다.

19. 설치, UI, 운영 runbook

처음 실행은 observe-only다. 관리 인터페이스·기존 nft/firewalld/Docker/CNI·route·MTU·IPv6·offload·writer 소유권을 점검한다. 설치 전에 OOB 콘솔 또는 동등한 복구 경로를 확보하고 좁은 baseline을 검증한다. 서명된 release를 설치하고 1회 enrollment token·target fingerprint·CSR로 agent를 등록한다. 기본 enrollment token 10분, mTLS 인증서 24시간은 운영 목표값이며 폐쇄망은 별도 프로필을 둔다.

UI는 채팅보다 실제 집행 상태를 먼저 보여준다. 필수 화면은 Overview, Requests, Plan Review, Policies/Leases, Assets/Bindings, Delegations/Envelopes, Deployments, Evidence, Integrations, Administration이다. 다음 정보를 숨기지 않는다: 마지막 관측 시각, desired/observed 차이, 새로 열린/닫힌 흐름, 남은 TTL, UNKNOWN, 검증 제외 범위, 다른 허용의 잔존 여부.

승인 버튼은 설명에 동의하는 버튼이 아니라 plan hash에 대한 결정이다. 만료와 검토기한을 같은 열의 expires로 표기하지 않는다. SSE 연결이 끊겨도 polling으로 조회 가능해야 하며 오래된 데이터를 실시간 초록 상태로 남기지 않는다. 키보드·색 이외 상태 표시·server-side pagination을 제공한다.

상황	즉시 조치	재개 조건
관리 연결 상실	watchdog 확인, OOB 진입	baseline·새 관리 연결 probe 통과
drift	해당 target 새 쓰기 중지	외부 변경 가져오기 또는 새 복구 계획 승인
signer 유출	새 grant 중지·키 철회·범위 조사	신뢰 root를 통한 교체와 target 재등록/검증
잘못된 deny	다음 wave 중지	해제 영향까지 승인된 복구 계획
DB restore	read-only 재연결	target 최고 generation·receipt와 조정
시각 불안정	새 임시 허용 중지	신뢰 시각과 잔여 lease 상한 확인
디스크 포화	저우선 로그 제한·경보	감사 journal의 durable 용량 회복
모델 오답 증가	해당 provider routing 중지	고정 평가 dataset 회귀 통과

break-glass는 사전 정의한 물리/OOB 경로와 제한 계정을 사용한다. API body의 긴급 문자열로 승인 경계를 우회하지 않는다. 사용과 실제 변경을 감사하고 사후 재조정한다. backup은 DB와 artifact를 같은 논리 checkpoint로 보관하고 실제 복구 drill을 수행한다. Team 목표 RPO 15분·RTO 4시간은 검수 후에만 운영 약속으로 제시한다.

20. Rust 구현 구조와 개발 계약

```
repository/
  crates/domain          타입·정규화·상태 전이
  crates/resolver        snapshot 기반 해석
  crates/validator       가드레일·집합 의미·coverage
  crates/compiler-nft    구조화 artifact 생성
  crates/controller      API·DB·outbox·application service
  crates/agent-access    신원·binding·봉투·위임·quota
  crates/mcp             protocol codec와 도구 변환
  crates/release         승인 재검증·grant 서명
  crates/agent           outbound 채널·저널·관측
  crates/enforcer        특권 집행·독립 watchdog
  crates/cli             사람이 사용하는 동일 서비스 client
  crates/adapter-sdk     순수 변환과 IO 계약
  web/                   정적 UI
  adapters/              선택적 연동 프로세스
  schemas/               생성·검증되는 wire 계약
  tests/                 단위·차분·netns·chaos·client
  examples/              합성 3-netns lab
  docs/                  설계·ADR·runbook·지원표
```

도메인 코어는 forbid(unsafe_code)를 적용하고, 검증 생성자와 역직렬화 경로를 일치시킨다. PortRange·CIDR·Timestamp·Digest·Generation·PrincipalId를 임의 문자열/정수로 서로 바꿔 쓰지 않는다. serde(try_from=...) 등으로 wire 입력도 같은 검증을 거치게 한다. typestate는 프로세스 안의 호출 순서를 돕지만 역직렬화된 Approved 문자열을 실제 승인으로 믿을 근거가 아니다.

AI > enforcer/release/장비 credential, compiler > AI/network, enforcer > LLM 등의 금지 의존 간선을 cargo metadata의 전이 의존 그래프로 검사한다. 운영 경로의 unwrap/expect·무한 spawn·무제한 channel/cache·무기한 child process는 금지한다. 작업별 deadline·취소·동시성 semaphore·stdout/stderr 상한을 둔다.

분석 API는 Result<Analysis, BudgetExceeded/Unsupported/Invalid>처럼 불확실성과 자원 실패를 표현해야 한다. BDD node_count를 나중에 읽는 것만으로 allocation 중 메모리 폭주가 예방되지 않는다. timeout을 빈 permit 집합이나 성공으로 바꾸지 않는다.

최종 JSON 계약은 구현 언어와 독립된 wire 정본이다. Rust 타입 생성/검증 파이프라인 또는 동등한 drift test로 맞춘다. Rust가 정본이라고 선언하고 실제 generator 없이 손으로 서로 다른 schema를 유지하지 않는다. 변경 PR은 의미 문서·schema·정상/반례 fixture·해시·호환성 계획을 함께 변경한다.

이 패키지의 Python 참조검증기는 계약 이해와 반례 재현 도구다. Rust 운영 코드의 대체물이 아니며 credential 검증·실제 DSSE 서명·원자 quota·packet enforcement 구현으로 취급하지 않는다. 제공된 Plan wire schema는 add_access_lease 요청의 변경 계획이다. requested_policies는 추가할 접근 권한만 나타내며 전체 target ruleset을 대체하는 명령이 아니다. backend는 baseline의 기존 allow/deny를 보존해 전체 candidate를 합성하고 전체 desired-state/artifact hash·차분을 검증해야 한다. 참조검증기는 그 실제 장비 합성까지 수행하지 않는다.

일반 deny 생성·삭제·baseline 변경을 같은 add-access payload로 실행하지 않는다. 이 작업들은 본문의 별도 승인·전체 차분·복구 의미를 따른 새로운 change_kind 계약으로 구현한다. ApprovalSet·PBOM·자산/binding registry·전체 후보 artifact의 완전한 wire schema도 구현 단계의 별도 산출물이며, 포함되지 않은 계약을 제공한 것처럼 표시하지 않는다. scope가 작은 개발 참조물을 보안 시스템 전체 구현으로 오인하지 않도록 계약 README와 검증 영수증에 이 범위를 고정한다.

21. 검증 계획과 성능·자원 목표

시험	반드시 포함할 반례	통과 조건
Parser/schema	중복 키·알 수 없는 필드·역전 port·family 불일치·큰 정수	명시 거절, panic·무한 처리 없음
의미 차분	겹치는 allow, deny 우선, 순열, 시간 경계	reference·compiler 판정 일치
실제 kernel	TCP/UDP 동일 port, IPv4/IPv6, return flow, fragment, PMTU	허용·차단·기존 세션 결과 일치
수명	A 만료/B 유지, 전체 만료, revoke, daemon kill	현재 P(t)와 패킷 결과 일치
변경 원자성	주소 축소·deny 추가·commit 중 kill	옛 mark/conntrack로 과다 허용 없음
시각·재시작	역행·큰 보정·부팅·VM restore·suspend 조건	만료 권한 부활 없음
승인	승인 후 artifact/target/commit 변경·review 철회	STALE/권한 거절
fencing	이전 generation·동일 세대 다른 내용·늦은 writer	재생·상태 덮어쓰기 거절
crash/UNKNOWN	nft 반영 직후 DB 실패·API timeout·confirm 유실	read-back 전 중복 쓰기 없음
rollback	이전 allow가 이미 만료·새 deny 유지 필요	승인된 복구 상한 준수
신원·quota	공유 IP·자산 사칭·동시 요청·다중 principal	표시한 보안 범위와 root 예산 준수
MCP	active/null·succeeded/unknown·잘못된 audience·retry	상태·payload·auth 계약 위반 거절
AI	없는 자산·거짓 span·prompt injection·유출 URL	권한 확대·비밀 접근 불가
복구/공급망	오래된 DB·철회키·변조 artifact·untrusted PR	새 권한 중단·재조정·실행 거절

항목	초기 측정 목표	측정 조건
controller RSS	idle 256 MiB 이하, cap 512 MiB	모델·DB 서버 제외, 정책 1,000건
agent RSS	정상 64 MiB, peak 128 MiB 이하	실제 kernel 메모리는 별도 보고
일반 API	p95 200 ms 이하	동시 20 client, 1,000정책
native 계획	p95 5초 이하	1 target·1,000정책, AI/외부 분석 제외
apply+read-back	p95 2초 이하	probe·승인·watchdog 대기 제외
종료 효과	만료 경계 이후 허용이 남지 않음	계측 오차·시계 오차 공개, 종료 후 1초 검사 목표
처리량	baseline 대비 저하 10% 이하를 초기 목표	64/512/1500 byte·PPS·IPv4/6·latency 공개
장기 실행	7일 고정 부하 후 bounded plateau	warm-up 이후 RSS 증가 5% 이하, slope·kernel·FD 별도

상한은 검증 예산과 함께 둔다: 일반 API 동시 64, Lite 작업 4, 대기 작업 1,000, AI provider 동시 1, 분석 worker wall time 2초·메모리 128 MiB부터 시작한다. 이 한도를 넘으면 admission control과 UNKNOWN/429를 사용한다. 모델·IDS·DB·conntrack·nft set의 메모리를 agent RSS 안에 숨기지 않는다.

문서에 기록된 과거 제한 시험은 해당 판본·환경·사례만의 증거다. 새 strict 합집합 의미, 새 schema, 새 복구 계약은 다시 시험해야 한다. 성능 표는 README 판매 문구가 아니라 측정할 조건이다.

22. 출시 순서와 구현할 첫 작업

일정은 인력·경험·외부 기여에 따라 달라진다. 달력만으로 운영 자동화를 승격하지 않는다. 1~2인이 13주 안에 코어·HA·NAT·모든 모델·멀티벤더를 완성한다는 약속은 하지 않는다. 단계의 출구 조건으로 진척을 판단한다.

단계	구현 결과	출구 조건
M0 계약	domain·schema·semantic evaluator·threat model	정상·반례·해시 정규화 시험
M1 packet lab	nft compiler·3-netns·수동 CLI	실제 dual-stack·응답·deny·중첩 만료
M2 복구	journal·watchdog·grant·generation·boot	kill·replay·clock·rollback 시험
M3 사용 흐름	SQLite controller·최소 UI·PBOM	AI 없이 요청 > 승인 > 적용 > 회수
M4 agent pilot	binding·봉투·root 위임·quota·MCP 6도구	두 client의 실제 end-to-end·동시 quota
M5 Team	PostgreSQL·OIDC·독립 signer·Git 검토	조직 격리·TOCTOU·restore·보안 검토
M6 확장	IDS·자산·모델·vendor read/diff	connector별 conformance와 maintainer
안정 v1.0	선택된 지원 프로필 고정	30일 운영·외부 검토·복구 drill·지원 책임

최초 공개는 design-preview 또는 실제 M1 이후 lab-preview로 명확하게 표시한다. UI는 M3에서 최소 요청·검토·상태 화면을 제공하므로 나중의 대시보드 완성판을 첫 설치가 의존하지 않게 한다. read/import 보고서는 기업 사용자가 쓰기 권한을 주기 전에 가치를 확인하는 별도 진입점이다.

첫 issue 묶음은 ①도메인 타입과 strict 의미 ②참조 평가기 ③nft compiler ④3-netns fixture ⑤kernel TTL·overlap ⑥저널·watchdog ⑦서명·fencing ⑧controller·outbox ⑨검토 UI ⑩PBOM ⑪binding·위임·quota ⑫MCP conformance ⑬supply-chain CI ⑭설치·복구 문서다. 실제 장비·외부 유료 계정이 없는 기여자도 앞의 코어와 fixture를 개발할 수 있어야 한다.

23. GitHub 공개, 기여와 Apache-2.0

공개 코드 저장소와 운영 정책 저장소를 분리한다. 공개에는 자체 작성 코드·계약·합성 fixture·문서만 넣는다. 실제 회사 자료·제출자 식별번호·IP·계정·장비 설정·고객 PBOM·모델 credential을 포함하지 않는다. 원자료를 사용자가 보유했다는 사실만으로 공개 재배포 권리가 확인된 것은 아니므로 저작권·비밀유지·회사 규정과 생성물 출처를 확인한다.

권리를 가진 자체 코어·계약·새 문서의 기본 라이선스는 Apache-2.0으로 채택한다. 사용·수정·배포 조건, 기여자의 일정 범위 특허 허여, 무보증·책임제한 조항이 명시돼 있다. 그러나 상업적 재사용·비공개 파생을 막는 라이선스가 아니며, 모든 소송·책임·제3자 권리 문제를 없애는 면책 장치도 아니다. 특허 보복 조항은 아무 비판이나 모든 분쟁을 금지하는 규칙이 아니다. 적용 법률이나 별도 계약상 책임이 우선할 수 있다. Apache-2.0 원문

Linux·nftables·Suricata·Grafana·모델·룰셋을 Apache-2.0으로 바꿔 표시하지 않는다. 별도 프로세스는 구조적 경계일 뿐 라이선스 의무 면제 증명이 아니다. nft CLI 사용과 GPL 라이브러리 직접 링크를 구분하고, 컨테이너/어플라이언스에 포함한 binary·수정본의 소스·고지 의무를 배포물별로 확인한다. XDP/eBPF를 추가할 때 GPL-only helper/kfunc·program type 요구를 별도 검토한다. 상세 근거는 라이선스 지침을 따른다.

기여는 DCO sign-off, 출처·권리 확인, 보안 영향·재현 시험을 포함한 PR로 받는다. DCO는 저작권 양도나 권리 침해의 면책 보장이 아니다. CLA가 필요한 기여·상업 모델이 생기면 목적을 공개하고 법률 검토로 별도 결정한다. 기여자마다 승인된 권리 이상을 주장하게 하지 않는다.

untrusted PR은 운영 secret·서명키·self-hosted production runner에 접근하지 못한다. 제3자 CI action은 고정 commit으로 잠그고, dependency·license·취약점·secret·금지 crate 간선 검사를 required job에 둔다. workflow에 없는 검사를 필수라고 문서에만 쓰지 않는다. 실제 maintainer 계정이 정해진 뒤 CODEOWNERS를 생성하고 각 줄은 한 pattern과 유효 owner로 검사한다.

release에는 source tag·binary/image digest·SBOM·provenance·license manifest·지원표·업그레이드/복구 절차를 포함한다. 보안 제보 경로와 지원 기간은 실제 운영 가능한 범위로 공개한다. Apache-2.0만 선택하고 유지보수·보안 문제를 끝낸 것으로 보지 않는다.

24. 최종 인수 기준과 남겨 둔 경계

최종 설계의 결론은 nftables 코어, 합집합 정책 의미, 엄격한 수명, 접근 봉투와 신원 위임의 교집합, 독립 서명, 관측 기반 상태 확정이다. 제품 확장은 같은 계약을 지킬 수 있는 범위에서만 진행한다.

출시 담당자는 다음을 확인해야 한다. AI·인터넷·GitHub가 없어도 기본 집행과 만료가 동작한다. 현재 허용 집합 밖의 기존 세션이 통과하지 않는다. 승인된 hash와 다른 대상·artifact·기간이 적용되지 않는다. UNKNOWN·drift·재부팅·시각 불신·부분 성공이 UI와 API에 드러난다. workload 수준 보장을 실제 네트워크 신원이 뒷받침한다. 외부 기여자가 lab을 재현하고 다른 maintainer가 release·복구를 수행할 수 있다.

최종 설계라는 말은 구현·운영에서 발견한 반례를 무시한다는 뜻이 아니다. 이 문서의 불변조건·정책 의미·위협 경계를 바꾸려면 ADR, 호환성 영향, migration, 새 반례 시험을 함께 제출한다. 지원되지 않는 기능·측정되지 않은 수치·확인되지 않은 법률 효과를 확정된 보장으로 바꾸지 않는다.

25. 출처, 입력 판본과 검증 범위

입력 자료의 파일 해시·ZIP 중복·PDF 페이지·tmp 분류는 F_입력자료_검토목록.json에 기록한다. 두 비교평가서와 과거 검증 영수증은 작성 당시 판본의 증거다. 현재 더 길어진 원문에서 해결된 지적을 그대로 현재 결함으로 옮기지 않았다. 최신 원문의 제한된 namespace 시험 보고도 원문 진술과 제공된 실행 로그 확인 여부를 구별했다.

공식 자료는 외부 기능의 범위를 확인하는 근거이고, 이 문서의 TTL·quota·승인 인원·단계·메모리 목표는 프로젝트가 선택한 설계 값이다. 문서 확인일은 2026-10-30이며 실제 구현 릴리스에서는 지원 버전의 문서를 다시 고정한다.

공식 근거	사용한 범위
nftables atomic replacement	단일 kernel transaction 범위
nftables element timeout	timeout과 논리적 만료를 활용하는 설계
nftables conntrack 표현	original/reply tuple·protocol·port·direction 표현
Suricata IPS 구성	버전이 고정된 NFQUEUE·inline 경로 문서; 제품 지원 버전 지정은 아님
MCP 2026-07-28 발표	새 protocol과 SDK·이전 규격의 구분
MCP authorization	resource 인증 경계
JCS RFC 8785	JSON canonicalization 명세 형식
DSSE	typed payload 서명 envelope
Apache-2.0	재배포·특허·상표·무보증·책임제한
Linux BPF licensing	후속 BPF 구성의 GPL 조건

F_검증결과.json에는 이번에 실제 수행한 문서·계약·도표 확인과 수행하지 않은 runtime 시험을 분리해 기록한다. 이 패키지에는 기존 모델 이름을 접두어로 한 산출물을 새 최종본으로 넣지 않는다. 최종 deliverable의 이름은 모두 F_로 시작하며, GitHub 표준 파일명은 공개 준비의 명시적 매핑에 따라 적용한다.

부록 A. 설계 결정과 통합 근거

문서 상태: 최종 설계 기준 / 작성일: 2026-10-30 / 판: 1.0

이 문서는 두 설계안, 상호 비교평가, 개발 계약과 검증 기록에서 무엇을 채택하고 무엇을 바꿨는지 설명한다. 제품의 규범적 동작은 함께 제공하는 최종 아키텍처 설계서를 따른다. 여기서의 '채택'은 설계 결정의 확정이며, 구현·성능·실장비 검증을 마쳤다는 뜻이 아니다.

1. 판단 원칙

출발점은 모델 간 우열이 아니라 같은 요청을 끝까지 안전하게 처리할 수 있는가다. 자연어 요청, 승인, 정책 배포, 패킷 처리, 만료, 장애 복구가 서로 다른 의미를 사용하면 각 부분이 그럴듯해도 전체 제품은 안전하지 않다. 따라서 다음 순서로 판단했다.

1. 실제 패킷과 권한의 의미가 명확한가.
2. 승인받은 내용과 실제 실행이 같은 대상·범위·기한에 결합되는가.
3. 중단·응답 유실·재시작·시계 변화 때도 그 의미가 유지되는가.
4. 무엇을 검증했고 무엇을 검증하지 못했는지 구분되는가.
5. 한 명의 유지관리자가 시작하여 공개 기여를 받을 수 있는 범위인가.

분량이 많다고 구현이 완성된 것은 아니며, 범위가 작다고 원래의 제품 목표를 모두 충족한 것도 아니다. 작은 집행 핵심과 넓은 확장 구조를 함께 보존하되, 확장 기능마다 실제로 쓰기를 허용하는 진입 조건을 둔다.

2. 자료의 버전과 증거 수준

2.1 비교평가서를 최신 원문보다 우선하지 않는다

과거 정밀 비교평가와 tmp/GPT_review/GPT_implementation_test_receipt.json이 검토한 Claude 원문의 SHA-256은 아래의 구판 값이다. 이번에 제공된 현재 원문은 다른 해시를 가진다. 따라서 과거 평가서의 행 번호와 결함 판정을 현재 원문에 그대로 옮기지 않았다.

자료	SHA-256	취급
현재 Claude 설계 원문	f00e27207d4cad1590cfc17604545ebe97ab08c1a83535208774ca7b76400004	최신 설계 의도와 현재 계약의 기준
과거 평가가 대상으로 삼은 Claude 원문	4a51a19ffee21448f4170e19a27fbdaa438cac18130897a50b2988384a80c1d1	당시 판정의 범위를 식별하는 값
현재 GPT 설계 원문	662d83577d68cf84df096b5816f3a30c2bb22dbdf85067a935545f6768e00d05	실행 계약과 제품 범위를 대조하는 기준

문서 내 C는 Claude_Rampart_오픈소스_AI통제형_방화벽_최종아키텍처_설계서.md, G는 GPT_오픈소스_방화벽_최종_아키텍처_설계서.md를 가리킨다. 이 표기는 출처 식별용이며, 결과물의 명칭이나 제품 이름이 아니다.

2.2 기존 시험 기록을 과장하거나 지우지 않는다

기존 검증 JSON은 스키마·예제·문서 검사와 추출한 Rust 인터페이스의 컴파일·일부 테스트를 기록한다. 과거 구현 검토 영수증에는 7개 Rust 테스트 통과가 기록되어 있고, 동시에 제품 빌드·패킷·벤더 시험은 아니라는 범위가 명시되어 있다.

현재 C 부록 K의 12번 항목은 Linux 5.15 및 nftables 1.1.3 환경에서 연결 표시 타임아웃, 절대 시각 만료, 충분한 잔여 시간의 세 가지 netns 시험을 수행했다고 보고한다. 두 만료 사례는 8초 뒤 차단, 유지 사례는 20초 지속으로 적혀 있다. 이는 원문에 보고된 제한된 시험 결과로 인정한다. 제공된 tmp의 관련 Markdown·JSON·셸·로그 파일에서는 이 최신 시험의 실행 스크립트와 원로그를 확인하지 못했다. 그러므로 독립 재현 완료나 전체 stateful 방화벽 검증 완료로 확대하지 않는다. 같은 부록의 bridge 시험은 커널 모듈 부재로 수행하지 못했다고 명시되어 있다.

최종 패키지의 검증도 '문서·계약 검사', '실행 가능한 참조 모델 검사', '커널 패킷 시험', '실장비 시험'을 각각 표시한다. 어느 하나의 통과를 다른 단계의 통과로 바꾸지 않는다.

3. 두 설계에서 보존한 가치

관점	보존할 내용	최종 설계에 반영하는 방식
독립 방화벽 제품	C의 단일 박스, 호스트·게이트웨이, 기존 장비 가져오기, 운용 화면, 장비별 기능 차이	네이티브 방화벽을 첫 집행 대상으로 확정하고, 단일 박스 설치 및 확장별 적합성 관문을 유지한다
변경 안전성	G의 계획 해시, 실행 권한의 대상·기한 결합, UNKNOWN, 안전한 복구, 단일 writer	대상 하나의 승인·적용 계약을 기본 단위로 삼는다
AI 운영	C의 내부 통역 AI·외부 요청자·운영 일꾼 구분, 직무와 작업 결과	역할별 권한을 분리하고 조사·정리·감사 일꾼은 형식화된 제안만 제출한다
에이전트 접근	양쪽의 사전 승인 봉투, 위임 범위, 짧은 허용, 예산	등록 주체·워크로드 결합·범위 교집합·독립 서명·원자 예산을 하나의 계약으로 묶는다
정책 의미	G의 차단 우선 및 허용 합집합	사용자 지정 priority를 v1에서 제거해 검증기와 집행기가 같은 의미를 사용한다
증적	양쪽의 정규화 JSON·다이제스트·서명·PBOM	계획·승인·실행 권한·적용 영수증을 순환 참조 없이 연결한다
고장 중 만료	C의 커널 이중 만료와 G의 재부팅·복구 시 재검증	현재 유효 권한을 매 패킷 검사하고, 커널 만료·신뢰 시각·재개 제한을 함께 규정한다
개발 구조	양쪽의 Rust 코어·TypeScript UI·모듈 경계	논리 모듈은 분리하되 처음부터 모든 모듈을 별도 서비스로 만들지 않는다
공개 운영	양쪽의 보안 정책·기여 지침·SBOM·서명·지원 등급	GitHub 공개 준비와 실제 지원 능력 표시를 릴리스 조건에 포함한다

4. 최신 원문에 대한 재판정

이전 쟁점	현재 원문의 상태	최종 판단
established 선허용으로 임시 연결이 계속 살아남음	C § 15.2는 strict 이중 만료, 연결 표시 판정표, flowtable 금지를 추가했다	이전 결함이 그대로 남았다고 평가하지 않는다. 다만 주소 축소 후 정리 지연과 겹친 허용의 세션 의미를 추가로 정리한다
FQDN이 다른 객체와 겹치지 않는다는 가정	C § 15.13은 실제로 차단하는 대역을 제외한 주소 전체의 과대근사로 바뀌었다	서로소 가정은 폐기되었다. deny 조건에도 같은 과대근사를 쓰는 문제는 별도이며, v1에서 동적 FQDN을 제외한다
deny 삭제가 무조건 Low	C § 12.1·12.6은 Δ+로 변경 위험을 계산하고 허용 확대의 자동 병합을 금지한다	해결된 의도를 채택한다. 차단의 검토 시각과 자동 종료 시각은 별도 필드로 정리한다
timeout 뒤 무조건 재시도	C § 14.5는 UNKNOWN, 영속 일지, 재조회, 대상 동결을 명시한다	보완을 인정한다. 대상 장비의 원자 fencing 여부는 추가로 구분한다
관리 연결의 생존만 확인	C § 15.3은 관리 PC의 신규 TCP 탐침, nonce, 별도 서명키를 명시한다	강점을 채택한다. production 관리 경로 변경에서 탐침 생략을 점수만으로 허용하지 않는다
DB 복구 후 과거 세대 재사용	C § 7.3·14.5는 장비·증적과 최대 epoch/seq를 비교한다	보완을 인정한다. 확인되지 않은 대상은 복구 상태로 유지하고 자동 쓰기 재개를 금지한다
CODEOWNERS가 다중 승인을 보장한다고 오해	C § 12.4는 한 줄 패턴, 오류 조회, 소유자 중 한 명 충족과 별도 인원 검사를 설명한다	해결된 계약을 채택한다
bridge 부팅 전에 전달 가능	C § 15.4는 LKG 적재 뒤 포트를 올리도록 정한다	의도를 채택한다. bridge는 실제 커널·L2 시험 전까지 지원하지 않는다
너무 짧은 전체 일정	C § 25는 v0.1 13주, 전체 순차 마일스톤 53주 이상으로 바뀌었다	과거 일정 수치를 재사용하지 않는다. 현재도 최초 범위가 커서 자체 포지·인가 서버·DNS·HA를 첫 릴리스의 필수품에서 분리한다
activation과 주체·바인딩의 결합 부족	C § 13.5·19.5·A.7은 principal, 바인딩, 독립 signer, 합산 예산을 추가했다	큰 보완을 인정하고 채택한다. 프로토콜을 빠뜨린 집합 키와 중복 grant 소유권은 다시 정의한다
작업 큐의 오래된 worker 완료	C § 19.6·A.10은 임대와 만료는 있지만 완료 요청의 claim epoch/token을 정의하지 않는다	현재도 계약 공백이다. CAS와 claim generation을 최종 계약에 추가한다
confidence를 보안 증거로 취급	C § 10.6은 confidence가 보정된 정확도나 보안 증거가 아님을 명시한다	보완을 인정한다. 공급자·모델·질문 유형별 평가 기록을 요구한다
schema·Rust 예제의 통과가 제품 완성 증거	양쪽 모두 일부 계약·시험 산출물은 있지만 완제품 검증과는 범위가 다르다	최종 설계에서는 계약 참조물과 구현 완료 상태를 분리한다

5. 확정한 설계 결정

D01. 정책 의미는 순서 없는 허용 합집합에서 차단 합집합을 뺀다

업무 흐름의 허용은 Permit(t) = Union(유효 allow, t) - Union(유효 deny, t)로 정의한다. 별도 보호되는 관리·네트워크 제어 baseline은 이 업무 규칙 집합과 구분한다. 요청자는 priority를 바꾸어 차단을 우회할 수 없다.

C의 first-match 모델은 기존 벤더 정책을 손실 없이 가져오는 데 유리하다. 그러나 신규 제품의 v1에는 사용자가 바꿀 수 있는 priority, 앞선 deny 목록, shadow 분석, 기존 세션의 rule ownership까지 함께 구현해야 하는 비용이 생긴다. G의 단순 모델을 선택하고, 기존 first-match 설정은 읽기·분석 전용으로 가져온다. 쓰기 대상으로 전환하려면 독립 의미 비교가 통과해야 한다.

검수: 입력 규칙의 순서를 바꿔도 허용 집합·판단·산출물 의미가 같아야 한다. 겹치는 allow가 있어도 deny는 우선한다.

D02. 세션 권한은 현재 허용 집합으로 판정한다

한 흐름에 A와 B가 동시에 권한을 주면 A의 만료만으로 B의 유효 권한까지 사라지지 않는다. 마지막 유효 allow가 끝나거나 deny가 적용되면 이후 패킷은 차단한다. 회수 결과에는 다른 유효 권한이 남는지를 표시한다.

이 선택은 G의 '특정 policy/version에 연결을 귀속하고 자동 승계하지 않음'과 C의 '첫 일치 규칙 표시로 유지' 양쪽을 수정한다. 하나의 연결이 어떤 허용 규칙을 우연히 먼저 만났는지에 따라 종료 시각이 바뀌는 문제를 없앤다. 업무별 연결을 반드시 구분해야 하는 경우에는 실제 워크로드 격리나 별도 네트워크 식별자가 필요하다.

conntrack mark는 현재 정책의 의미를 보존하는 캐시일 수 있다. mark 자체를 영구 승인으로 사용하지 않는다. 정책 변경은 활성 버전 및 캐시 유효성을 원자적으로 바꾸고, 후속 패킷은 현재 정책으로 재검사한다. 주소·포트·인터페이스 축소 후 conntrack 삭제가 완료되기 전까지 구 권한이 통과하는 구간을 허용하지 않는다. conntrack 삭제는 정확성의 전제보다 후속 정리 작업으로 둔다.

검수: A/B 중첩, A 회수, B 만료, deny 추가, 주소 축소, 규칙 재정렬, 에이전트 종료를 조합한 양방향 TCP·UDP 시험을 한다.

D03. 상태 기반 방향과 패킷 방향을 혼동하지 않는다

업무 정책의 방향은 연결을 시작한 방향이다. 반환 패킷을 반대 방향의 신규 접근으로 다시 해석하지 않는다. 원래 방향의 주소·서비스 튜플, IP family, protocol, 신뢰되는 인터페이스/영역, conntrack direction을 함께 사용한다. NAT를 지원하지 않는 v1의 범위를 분명히 유지한다.

일반적인 established,related accept는 사용하지 않는다. 자동 conntrack helper를 끄고, 관련 ICMP 오류·PMTUD·필수 IPv6 제어 통신은 별도 명시적 프로필과 시험으로 다룬다. 예상하지 못한 비대칭 경로나 워크로드 식별 실패는 더 넓은 허용으로 바꾸지 않는다.

D04. allow 만료와 deny 검토를 다른 계약으로 둔다

임시 allow의 expires_at은 집행 종료 시각이다. deny의 review_at은 사람이 재검토할 시각이며 자동 차단 해제 시각이 아니다. v1에서는 deny의 자동 만료·삭제를 지원하지 않는다. 제거 요청은 새 계획의 Δ+를 검증하고 승인받는다.

C의 on_expiry: review/remove는 차단 해제의 위험을 잘 드러냈다. G의 deny expires_at 의미 보정도 같은 위험을 인지했다. 최종본은 필드 자체에서 의미를 분리해 독립 SDK나 UI가 잘못 해석할 가능성을 줄인다. 자동 대응 차단의 종료는 후속 RFC에서 시작·종료를 함께 승인한 별도 수명주기로 정의한다.

D05. 필수 검증의 UNKNOWN은 승인 점수로 덮지 않는다

검증 결과는 PASS, FAIL, UNKNOWN, NOT_APPLICABLE을 구분한다. 필수 검사가 UNKNOWN 또는 모델 밖이면 그 계획은 배포할 수 없다. 위험 등급 상승과 추가 승인으로 정확성 부재를 대체하지 않는다.

C §11.5의 BDD 예산 초과를 보증 공백 점수로 올리는 방식은 검증 범위를 넓힐 수 있다는 장점이 있지만, 불변식의 성립을 확인하지 못한 상태가 쓰기 허가로 바뀔 수 있다. 최종본은 지원 범위를 좁히거나 검증을 다시 수행해 PASS를 얻도록 한다. 비필수 참고 분석의 미실행은 별도 coverage로 공개할 수 있다.

D06. 노출 위험과 중단 위험을 함께 승인한다

Δ+는 새로 열리는 흐름이고 Δ-는 닫히는 흐름이다. Δ+가 비었다고 업무 영향까지 작은 것은 아니다. deny-all, 중요 서비스 차단, 관리 경로 변경은 노출을 줄이면서 큰 장애를 만들 수 있다.

양쪽의 위험 점수는 설명과 우선순위에 사용한다. 정책 위반·검증 실패·관리 경로 보호·핵심 업무 중단 조건은 점수와 별도 강제 조건이다. 안전한 자동 회수는 만료된 allow의 정리 등 좁은 경로에서만 허용한다.

D07. 하나의 계획은 하나의 집행 대상을 가진다

최소 실행 단위는 target 하나, 그 target의 승인된 baseline 하나, 후보 artifact 하나다. PlanContent의 해시는 스냅샷·정규화 IR·도구 버전·검증 결과·영향·기한과 결합한다. 승인과 실행 권한은 이 해시를 참조한다.

여러 대상의 작업은 별도 workflow가 대상별 계획과 상태를 모은다. 전체 성공은 필요한 모든 대상의 영수증으로 확인하며 일부 성공을 전체 ACTIVE로 표시하지 않는다. 여러 장비와 DB를 하나의 원자 트랜잭션이라고 부르지 않는다.

D08. UNKNOWN 상태에서 재시도보다 관측을 먼저 한다

장비가 쓰기를 처리한 뒤 응답이 유실되면 실패로 단정하지 않는다. 먼저 영속 일지와 장비 재조회로 적용·미적용·부분 적용을 구분한다. 불확실한 동안 동일 쓰기를 다시 보내거나 관련 예산을 반납하지 않는다.

양쪽 최신 계약의 일지·단일 writer·재조회 원칙을 채택한다. 복구도 현재 시점의 유효 권한만 복원하며 이미 만료·철회된 allow를 되살리지 않는다. 복구 후 패킷 손실이나 외부 서비스의 부작용까지 취소된다고 주장하지 않는다.

D09. lease와 실제 fencing을 구분한다

네이티브 집행기는 자신이 저장한 epoch/generation보다 오래된 명령을 실제 적용 경계에서 거부한다. DB lease만 보유했다는 사실로 충분하다고 보지 않는다.

C §14.5의 벤더 주석 rp-gen 선조회 후 쓰기는 유용한 충돌 탐지지만 compare-and-swap이 아니다. 구 작성자가 낮은 값을 읽은 뒤 멈췄다가 새 작성자의 적용 뒤 쓰면 이전 정책을 덮을 수 있다. 장비가 제공하는 잠금·조건부 변경·세션 fencing을 검증하지 못한 어댑터에는 네이티브와 같은 강한 동시성 보장을 붙이지 않는다. 해당 관리 범위에 direct API와 외부 관리 도구를 동시에 writer로 두지 않는다.

D10. 관리 접근 확인은 실제 신규 연결로 한다

기존 agent 연결의 생존, 로컬 리스닝 포트, 설정의 lifeline 존재는 각각 유용하지만 원격 관리자의 신규 연결 성공과 같지 않다. production 관리 경로를 바꾸는 배포는 대표 관리 출발지의 새로운 연결 탐침을 요구한다. 탐침은 대상·계획·세대·일회 challenge에 결합한다.

C가 보강한 별도 탐침 키와 G의 신규 관리 연결 요구를 함께 보존한다. 로컬 watchdog은 적용 전에 시작한다. 확인이 없으면 유효한 안전 복구 상태로 돌아가며, 복구도 확인하지 못하면 대상 상태를 DEGRADED로 남긴다.

D11. 시각·재부팅·재개를 같은 만료 계약에 포함한다

커널 상대 timeout과 신뢰 UTC에 따른 절대 기한은 서로 다른 고장을 막는다. 하나만으로 모든 환경의 정확한 기한을 보장하지 않는다. 신뢰 시각을 확인하지 못한 부팅에서는 임시 허용을 복원하지 않는다.

v1의 인증된 집행 프로필에서는 suspend/hibernate 및 live snapshot restore를 지원하지 않는다. 해당 기능을 추가하려면 재개 첫 패킷 이전의 격리, 시각·권한 epoch·만료 재검증을 실제 환경에서 입증해야 한다. '재개 후 사용자 공간이 언젠가 탐지'하는 방식으로 절대 만료를 보장하지 않는다.

D12. AI 권한 경계는 프로세스·자격증명으로 강제한다

내부 통역 AI는 DraftIntent를 만들고, 외부 에이전트는 요청이나 제안을 낸다. 둘 다 장비 쓰기 자격증명과 승인 권한을 갖지 않는다. 독립 signer는 controller의 approved=true 같은 DB 값을 그대로 믿지 않고 승인 대상과 권한 근거를 검증한다.

C의 독립 signer, 공급자 프로세스 격리, Laya 식별과 작업별 평가 접근을 채택한다. G의 공급자와 에이전트 호스트 구분, 제한된 호출 표면도 유지한다. 특정 브랜드나 typed output·confidence를 보안 판정의 근거로 사용하지 않는다. 구독 CLI 연결은 도구 실행을 사전에 제한할 수 있는 검증된 경로에서만 사용한다. read-only나 사후 결과 폐기는 무도구 실행 보장이 아니다.

D13. 봉투는 범위 상한이며 인증·위임·예산을 대신하지 않는다

즉시 허용의 조건은 승인된 봉투, 등록 주체, 현재 워크로드 결합, 위임 범위, 조직 가드레일, 잔여 예산의 교집합이다. 요청의 출발지는 인증 주체의 검증된 바인딩에서 정하며, 요청 본문으로 바꾸지 않는다.

v1은 등록된 루트 주체의 단일 위임 범위를 지원한다. 하위 에이전트의 동적 생성과 재위임은 후속 단계로 둔다. 확장 시 부모·조직 예산은 자식 수가 늘어도 증가하지 않으며 권한과 만료는 부모 범위 안으로만 줄어든다.

서명된 activation에는 principal, workload binding revision, 봉투/위임 digest, reservation, target, protocol, 절대 기한, 재생 방지 식별자가 결합된다. 독립 서명 및 집행 경계에서 다시 확인한다.

D14. 동적 허용의 키와 소유권을 완결한다

C § 19.5의 예시 JIT 집합은 (src,dst,port)라 protocol이 없다. 동일 주소·포트의 TCP와 UDP가 봉투에 함께 있으면 한쪽 activation이 다른 쪽 정적 규칙에도 사용될 수 있다. 최종 계약의 흐름 키에는 적어도 IP family, protocol, source, destination, service와 집행 범위를 포함한다.

같은 흐름에 여러 grant가 있으면 단일 집합 원소 삭제로 소유권을 구현하지 않는다. 승인 원장은 grant별 권한과 기한을 보존하고 집행기는 현재 유효 grant들의 합집합을 투영한다. 한 grant를 철회해도 다른 grant를 지우지 않으며, 마지막 권한이 끝나면 원소와 후속 패킷 허용이 끝난다. 만료를 갱신할 때 과거 메시지의 재전송으로 기한이 늘어나지 않는다.

D15. 네트워크 식별자가 없으면 에이전트별 패킷 격리를 주장하지 않는다

MCP 토큰 검증은 요청자 확인이다. 같은 IP·network namespace·NAT 출발지를 공유하는 두 프로세스의 실제 패킷을 IP 방화벽이 구별하게 해 주지는 않는다.

C의 바인딩, L2 격리와 anti-spoof 요구를 보존한다. 격리된 VM/namespace, 신뢰되는 인터페이스·주소 할당, 재할당 방지 조건이 없으면 제품은 공유 네트워크 주체 단위의 허용만 보장한다. 주소 재사용은 남은 grant와 conntrack을 고려하여 지연하거나 격리한다.

D16. 운영 일꾼은 제안하고 오래된 작업 결과는 거부한다

드리프트 조사, 정리 후보, 탐지 규칙 초안, 감사 요약 등 C의 운영 직무를 유지한다. 결과는 정해진 schema에 맞는 데이터이며 자동 승인이나 직접 배포 명령이 아니다.

작업 claim에는 claim_generation과 불투명 token을 발급한다. heartbeat, update, complete는 현재 소유자·generation·유효 임대를 비교해 원자적으로 갱신한다. 오래된 worker의 완료와 파생 제안은 거부한다. 재시도 가능한 후속 요청은 작업 ID와 허용된 action revision에 결합한 멱등 키를 쓴다.

D17. MCP는 공통 애플리케이션 서비스를 운반한다

도구 schema는 외부 사용자 정의 URI를 해석해야만 동작하는 형태로 배포하지 않는다. wire schema는 필요한 정의를 내부에 묶는다. 성공·업무 거절·시스템 오류·비동기 상태의 닫힌 결과 형식을 제공하고, 페이지 조회 입력·출력 및 operation별 결과 형식을 함께 정의한다.

짧은 요청 안에 끝나지 않은 작업은 operation ID와 상태 조회 경로를 반환한다. timeout을 미적용으로 해석하지 않으며, 호출 취소와 이미 발급된 접근권한 회수를 구분한다. 사람이 이해하는 화면과 에이전트용 인터페이스는 같은 권한 검사와 상태 전이를 사용한다.

D18. 작은 실행 코어와 명확한 프로세스 경계를 유지한다

Rust 도메인 모듈, AI 프로세스, signer, 권한 있는 집행기를 분리한다. 그 밖의 접수·조회·작업 조정·관측 기능을 처음부터 모두 분산 서비스로 배포하지 않는다. SQLite는 단일 controller 프로필에 한정하고, 다중 controller는 PostgreSQL 및 별도 동시성 검수 뒤 제공한다.

자체 Git 포지, 자체 OAuth 인가 서버, DNS 리졸버, DHCP 서버, Wasm 플러그인 호스트를 첫 핵심 기능의 전제로 두지 않는다. 기존 도구와 최소 계약을 사용하고, 직접 구현해야 할 제품상 이유와 유지보수 인력이 생길 때 분리된 RFC로 추가한다. 단일 박스라는 제품 가치는 유지하되 같은 호스트의 여러 Unix 사용자 분리가 root 침해까지 막는다는 주장은 하지 않는다.

D19. 문서 계약과 출시 지원 상태를 분리한다

JSON Schema의 통과는 올바른 권한을 뜻하지 않는다. 타입 범위, 참조 무결성, 인증, 기한, 서명, baseline, 현재 대상 상태를 추가로 검사한다. 문서용 생략 부호를 실제 digest 값으로 허용하지 않는다. 예제는 실제 길이의 합성 digest를 사용하며, 런타임 검증과 참조 모델 검사를 별도로 보고한다.

‘지원 예정’, ‘읽기/렌더’, ‘계약 시험’, ‘실제 쓰기 검증’을 구분한다. nftables의 커널 트랜잭션 원자성을 모든 벤더나 여러 대상의 원자성으로 확대하지 않는다. 잘못된 기능을 더 넓은 allow로 근사하는 fallback은 금지한다.

D20. GitHub 공개는 재사용 권리와 실제 품질 약속을 분리한다

자체 작성 코드·문서·예제의 공개 라이선스는 최종 공개 지침에서 정한 Apache-2.0 정책을 일관되게 따른다. 외부 코드·규칙·모델·이미지의 라이선스는 별도로 보존한다. 코드의 서명·취약점 접수·지원 범위·증거 있는 릴리스 노트는 라이선스 선택과 별개 운영 책임이다.

기존 C의 문서 CC BY 4.0과 G의 다중 산출물은 그대로 혼합하지 않는다. 실제 기여물의 권리 상태를 확인하고, 최종 새 저작물의 표시를 정리한다. 라이선스가 보안 사고나 모든 법적 책임을 없앤다고 설명하지 않는다. 구체적인 보호 범위와 공개 전 권리 점검은 별도 라이선스 지침을 따른다.

6. 확장 목표와 쓰기 허용 관문

확장 목표	보존하는 제품 가치	쓰기를 허용하기 전의 필수 조건
NAT	가정·사무실 게이트웨이와 포트 포워딩	original/transformed tuple, route/filter 순서, 기존 NAT 세션, rollback, lifeline 보호를 별도 의미 모델과 패킷 시험으로 검증
FQDN egress	이름 기반 개발 서비스 접근	DNS snapshot과 갱신 권한, TTL 상한, rebinding, 공유 IP, 부정 조건의 건전한 상·하한 분석; hostname 강제와 IP 허용을 구분
Bridge	주소 변경 없는 삽입	L2·ARP·IPv6·브리지 conntrack, 부팅 전 격리, 루프·관리 경로 및 실제 모듈 적합성 시험
XDP	대량 공격의 조기 drop	DROP/PASS만 제공하고 PASS 뒤 nft 검사 유지; fragment·IPv6·응답 방향·보호 흐름·map 갱신·자원 상한 시험
Suricata inline IPS	허용 흐름의 콘텐츠 검사	정책 허용 후 별도 hook, 양방향 검사, 무리스너·queue 포화·재시작을 구분한 fail-open/closed, offload 우회 금지
HA	게이트웨이 장애 시 복구 시간 단축	정책 version·동적 권한·만료·conntrack mark 동기화, split-brain fencing, 시각, standby의 실제 경로 검수
벤더 쓰기	기존 투자와 공존	버전별 capability, 독립 importer, 장비 잠금·CAS, 부분 적용·미확정·세션 만료·보상 복구 실장비 시험
하위 에이전트	대규모 AI 작업 조직	위임 체인 진위, 감쇠 권한, 합산 예산, 동시 예약, root 정지 전파, 재생·재할당 시험
자동 대응	탐지에서 제한된 차단까지 단축	인증된 탐지 근거, 위조 출발지 배제, 보호 목록, 시작·종료 범위 사전 승인, 오탐 복구 시험
Wasm 플러그인	커뮤니티 어댑터	허용된 IO 중개, 비밀 비노출, 자원·호스트·경로 제한, 계획 밖 쓰기 차단, 독립 읽기 검증

Suricata 공식 문서는 nftables 방화벽 판정 뒤의 별도 IPS 체인을 설명하며 bypass는 queue listener가 없는 경우의 동작을 바꾼다. queue 포화까지 같은 실패로 단순화하지 않는다. Suricata NFQUEUE 구성

nftables flowtable은 일반 forwarding 경로를 우회하므로, 만료·재검증·IPS와 함께 쓰려면 그 빠른 경로 자체를 시험해야 한다. nftables 공식 매뉴얼

7. 채택하지 않은 주장

- Rust나 typed output만으로 논리 오류·메모리 누수·환각이 사라진다는 주장.
- BDD, 서명된 증적, schema 통과를 전체 네트워크의 형식 증명으로 부르는 표현.
- 날짜나 모델 버전이 적혀 있다는 이유만으로 외부 SDK·장비 기능이 검증됐다는 주장.
- source IP가 같아도 MCP principal이 다르면 패킷을 자동 구별할 수 있다는 주장.
- 라이선스만 선택하면 상용 재사용·상표 침해·보안 사고·제3자 권리 문제까지 해결된다는 주장.

• 문서나 비교점수의 우수성이 출시 준비 완료를 뜻한다는 주장.

8. 최종 검수에 연결할 반례

반례	실패하면 안 되는 동작
allow A가 만료되지만 B는 유효함	규칙 순서 때문에 B 권한까지 조기 종료하거나 A를 영구 연장하지 않는다
allow 객체의 주소 하나가 제거됨	conntrack 정리 전 구 mark로 해당 주소의 패킷을 계속 허용하지 않는다
TCP/53 grant만 있고 봉투에는 UDP/53도 있음	UDP/53이 함께 활성화되지 않는다
같은 흐름에 grant 두 개가 있고 하나만 회수	다른 유효 grant의 권한과 증적을 삭제하지 않는다
필수 Twin 계산이 예산을 초과함	높은 위험 점수나 사람 승인만으로 PASS로 바꾸지 않는다
target이 적용했으나 응답이 유실됨	동일 쓰기 재전송이나 예산 선반납을 하지 않는다
이전 writer가 멈췄다가 뒤늦게 재개	최신 target generation을 낮은 값으로 덮어쓰지 않는다
기존 관리 TCP는 살아 있지만 신규 연결은 차단됨	배포를 정상 확정하지 않는다
작업 A의 임대 만료 뒤 B가 점유	오래된 A의 완료·후속 제안을 받지 않는다
과거 snapshot을 만료 뒤 복원	이전 허용을 첫 패킷부터 다시 사용할 수 있게 하지 않는다
기존 설정에 순서 예외가 있음	순서를 삭제하고 의미가 달라진 union 정책을 동등하다고 표시하지 않는다
public FQDN deny를 넓게 근사함	실제 허용 상한을 줄여 계산하고 안전하다고 판정하지 않는다

이 반례들은 구현 수용 기준이다. 이번 최종 설계 산출물에서 해당 장비·커널 시험까지 수행했다는 의미로 읽지 않는다.

부록 B. 라이선스와 GitHub 공개

설계 기준판 1.0 | 2026-10-30 | 적용 대상: 통제형 오픈소스 방화벽 프로젝트

최종 결정은 자체 작성한 사용자 공간 코어·API·SDK·UI·스키마와 새 프로젝트 문서의 기본 라이선스를 Apache License 2.0으로 통일하는 것이다. 외부 코드, eBPF 모듈, 규칙, 모델, 데이터, 이미지·폰트에는 해당 구성요소의 조건을 따로 적용한다. 현재 결과물은 설계와 공개 준비 문서이며 실행 가능한 방화벽 제품, 실제 배포물의 라이선스 감사, GitHub 공개 완료를 의미하지 않는다.

1. Apache 2.0이 작성자를 보호하는 범위

Apache-2.0은 무보증·책임제한, 저작권·라이선스 고지 유지, 명시적 특허 허여를 제공한다. 다만 적용 법률이 요구하거나 별도 서면으로 약정한 책임에는 예외가 있다. 소프트웨어의 보안성·무중단·특정 용도 적합성을 보장하는 증명서도 아니다. Apache License 2.0, § § 2–9

항목	기대할 수 있는 효과	한계
저작권	라이선스 제공은 저작권 양도가 아니며, 관련 고지와 수정 사실을 보존하게 한다	다른 사람의 정당한 상업적 재사용을 막는 권한은 아니다
특허	기여자가 허락할 수 있는 일정 범위의 특허 청구항을 사용할 수 있게 한다	모든 제3자 특허의 무침해·분쟁 비용을 보장하지 않는다
특허 소송	해당 작업·기여에 대한 특허 침해 주장을 제기하면 정해진 특허 허락이 종료될 수 있다	모든 종류의 소송이나 책임에서 면제하지 않는다
제품 사고	무보증·책임제한 조건이 있다	강행 법률, 고의·중과실 등 법률상 예외, 별도 약정은 따로 고려한다
브랜드	상표 사용권을 일반적으로 허여하지 않는다	이름·로고의 실제 권리 확보와 별도 정책은 필요하다

위 표는 라이선스의 일반 구조를 설명한다. 개별 분쟁에서의 효력은 관할과 사실관계에 따라 다르다. 원문과 ASF 특허·배포 FAQ를 함께 확인한다.

타인이 코드를 가져가 상업 제품·비공개 파생 제품에 사용하는 것을 막는 목적이라면 Apache-2.0은 그 목적에 맞지 않는다. 수정 내용을 원 프로젝트에 돌려줄 의무도 일반적으로 요구하지 않는다. 이 프로젝트에서는 공개 개발과 도입 확대를 우선하고, 신뢰할 수 있는 릴리스·브랜드·지원·유지보수로 지속 가능성을 확보한다. ASF FAQ: 수정·재배포

개인 프로젝트가 Apache-2.0을 사용해도 Apache Software Foundation 소속·인증·법률 지원을 받는다는 뜻은 아니다. 이름에 Apache를 넣거나 공식 후원을 암시하지 않는다. ASF FAQ: 자체 소프트웨어 적용 및 상표

2. 라이선스 적용 경계

대상	최종 원칙	공개 시 증거
자체 사용자 공간 코어 ·CLI·API·SDK·UI	Apache-2.0	파일별 SPDX 식별자, 기여 이력, 루트 LICENSE
자체 작성 스키마·정책 예제·새 문서	Apache-2.0	제3자 자료의 원 조건을 별도 표시
외부 라이브러리·복사한 코드	각 원 라이선스	버전·소스·해시·LICENSE·NOTICE·수정 내역
Linux·nftables·Suricata 등 별도 프로그램	원 라이선스 유지	실제 배포 여부와 대응 소스 제공 방법
eBPF/XDP 모듈	프로그램 타입·helper에 맞는 GPL 호환성 별도 결정	소스 SPDX, 로더 license, helper·kfunc 목록
Grafana·기타 외부 서비스	서비스별 라이선스 및 약관	사용 에디션·수정·결합·배포 형태
모델 가중치·토크나이저·데이터·규칙 피드	코드와 독립적으로 확인	개별 자산의 권리·재배포·수정 조건
이름·로고·상표	소프트웨어 라이선스와 별도	권리자 확인과 상표 사용 정책

Apache-2.0 영어 원문을 수정하지 않는다. 비상업적 이용 제한, 경쟁사 사용 금지, 책임의 무조건 전가 등을 덧붙여 놓고 Apache-2.0이라고 표시하지 않는다. 번역은 설명용이며 원문을 대체하지 않는다. ASF FAQ: 라이선스 수정·번역

3. 방화벽 의존성과 배포 방식

3.1 Linux와 nftables

Linux 커널은 GPL-2.0-only와 명시적 syscall 예외를 사용한다. Linux 위에서 실행하는 독립 사용자 공간 프로그램이라는 사실만으로 자체 코어가 GPL이 되는 것은 아니다. 반대로 커널을 포함한 VM·어플라이언스를 배포하면 커널 자체의 배포 의무는 남는다. Linux kernel licensing rules

nftables는 공식 문서에서 GPLv2로 안내한다. v1은 nft를 **고정 실행파일과 분리된 인자 배열로 실행하는 외부 프로세스 방식**으로 한다. 셸 명령 문자열을 조립하지 않는다. libnftables를 코어에 직접 정적·동적 링크하거나 FFI로 연결하는 것은 다른 결합 방식이므로 v1에서 제외하고, 도입 전 별도 ADR와 라이선스 검토를 요구한다. 고수준 libnftables와 저수준 libnftnl도 구분한다. nftables 공식 manpage, Netfilter 라이선스 안내

Apache-2.0과 GPLv2-only의 직접 결합에는 호환성 문제가 있다. GPLv3와의 호환성은 결합물 전체를 Apache-2.0만으로 배포해도 된다는 뜻이 아니다. ASF GPL 호환성 설명

CLI·소켓·컨테이너 분리는 독립 프로그램이라는 판단에 도움을 주지만 자동 면제 규칙이 아니다. 공유하는 내부 구조와 통신의 결합 정도도 고려해야 한다. **프로세스 분리와 재배포 의무 이행을 별개로 검토한다.** GNU GPLv2 FAQ: mere aggregation

3.2 eBPF/XDP

Linux 공식 문서는 GPL-only helper, kfunc 및 일부 프로그램 타입에 GPL 호환 라이선스가 필요하다고 설명한다. Rust/Aya로 작성해도 이 요구는 동일하게 검토한다. BPF licensing

eBPF는 후속 독립 모듈로 개발한다. 실제 프로그램 타입·helper·가져온 소스를 확정하고 소스 라이선스와 로더 선언을 일치시킨다. 직접 작성한 코드에 GPL 호환 선택지를 두는 방식을 검토할 수 있지만, 제3자 코드까지 임의로 재라이선스하지 않는다. 로더 문자열만 GPL로 선언하는 것은 적합성 검토의 대체가 아니다.

3.3 Suricata·탐지 규칙

Suricata는 GPLv2이며 수정·라이브러리 결합·재배포에 따른 조건이 있다. 기본 연동은 독립 프로세스의 EVE JSON을 읽는 방식으로 한다. 엔진 코드 수정·링크 또는 자체 이미지 번들은 별도 배포 검토 대상이다. Suricata 라이선스 안내, OISF GPL FAQ

엔진과 탐지 규칙의 라이선스는 별개다. ET·Sigma·커뮤니티·상용 규칙의 이름만으로 재배포를 승인하지 않는다. 기본 배포에는 프로젝트가 권리를 가진 실습 규칙만 넣고 외부 피드는 이용자가 승인한 소스에서 선택한다. 각 규칙 번들의 출처·버전·수정·상업 이용·재배포·구독 조건을 기록한다.

3.4 관측·비밀관리·자산관리

Grafana의 핵심 오픈소스 프로젝트는 AGPLv3로 전환됐다. 정확한 에디션과 버전의 LICENSE를 확인한다. 수정된 AGPL 프로그램을 네트워크로 이용시키는 경우에는 § 13의 소스 제공 기회 요건을 검토한다. 단순 API 이용만으로 자체 코어가 무조건 AGPL이 된다고 하거나, 별도 컨테이너라는 이유로 항상 무관하다고 단정하지 않는다. Grafana licensing, Grafana LICENSE

OpenBao는 MPL-2.0, NetBox core는 Apache-2.0로 확인된다. 같은 브랜드의 플러그인·상용 애드온까지 조건이 같다고 추정하지 않는다. OpenBao LICENSE, NetBox LICENSE

나머지 Rust·UI·DB·검증 라이브러리는 실제 lockfile과 릴리스 파일에서 버전·전이 의존성·라이선스를 확정한다. 설계에 등장한 선택적 연동을 이미 포함된 의존성이나 검증 완료 지원으로 표시하지 않는다.

3.5 AI 모델·외부 API

모델 실행 코드, 기반 모델, 가중치, 토크나이저, 학습·평가 데이터, 상용 API 약관은 별도 승인 단위다. 예를 들어 Laya 저장소의 Apache-2.0 표시는 코드에 대한 확인 근거이지 임의의 파생 가중치와 데이터 전체에 대한 승인 증거가 아니다. Laya LICENSE

초기 저장소·배포 이미지에는 외부 가중치와 유료 SDK를 넣지 않는다. 선택 다운로드도 출처와 조건을 표시하며 계정·구독·접근 제한을 우회하지 않는다. 모델 선택과 데이터 송신 허가는 운영 정책으로 따로 관리한다.

4. 배포물별 필요한 자료

배포물	추가 확인 사항
자체 소스 저장소	파일 출처·SPDX·LICENSE·기여 권리·복사한 코드·문서 자산
자체 바이너리	정적·동적 링크한 라이브러리, 생성 코드, 포함 UI·폰트·Wasm
컨테이너	모든 레이어의 OS 패키지와 도구·런타임·모델·규칙
VM·어플라이언스	커널·부트 구성·드라이버·펌웨어·배포판 패치까지 포함
플러그인·규칙·모델 번들	각 번들의 원문·고지·재배포 권한·라이선스 호환성

GPL 등이 요구하는 대응 소스에는 정확한 바이너리 버전의 소스와 해당 빌드·설치에 필요한 스크립트가 포함될 수 있다. 실제 배포에 맞는 제공 방식을 선택하고 릴리스에서 접근 가능하게 유지한다. 단순 upstream 홈페이지 링크 하나로 의무 이행을 완료했다고 처리하지 않는다. Netfilter 배포 안내

릴리스 manifest의 필수 필드는 component_kind, source_url, version_or_revision, sha256, license_expression, license_text_path, copyright_notice, redistribution_allowed, modification_conditions, source_delivery, reviewer, reviewed_at다. SBOM·스캔 결과는 증거이며 법률 검토의 자동 대체가 아니다.

5. 원자료와 개인·회사 권리

현재 작업 자료가 채용 사전과제와 관련된 폴더에 있다는 사실만으로 공개 권한의 유무를 확정할 수 없다. **새 GitHub 저장소는 최종 독립 설계와 프로젝트가 권리를 가진 파일만 허용 목록으로 옮긴다. 작업 폴더 전체를 저장소에 올리지 않는다.**

공개 전 다음을 확인하고 확인자·일자·근거·허용 파일을 기록한다.

1. 회사가 제공한 과제 원문·안내 PDF·슬라이드·이미지·로고를 재배포할 권리가 있는가. 권리가 확인되지 않으면 포함하지 않는다.
2. 제출·채용·고용·외주 계약에 비밀유지 또는 지식재산 귀속 약정이 있는가. 이 문서만으로 해당 약정의 존재나 효력을 판단하지 않는다.
3. 실제 내부 주소·호스트명·조직도·네트워크 설정·로그·개인정보·토큰·인증서가 남아 있는가. 예제는 가상 데이터로 바꾼다.
4. 외부 코드·표·그림·긴 문서 인용을 복사했는가. 출처와 원 라이선스가 허용하는 범위를 확인한다.
5. 기존 문서의 CC BY 등 표기가 실제 권리자의 결정인가. 기존에 유효하게 적용된 타인 라이선스를 새 LICENSE로 덮어쓰지 않는다.
6. 본인이 검토·작성한 기여와 AI 보조로 생성된 부분의 출처를 추적할 수 있는가. AI 생성이라는 이유로 제3자 권리 검사를 면제하지 않는다.

본 최종본의 라이선스 결정은 권리를 가진 자체 작성분에 대한 것이다. 선행 자료 전체의 저작권을 프로젝트가 소유한다고 주장하지 않는다. 개별 계약·권리·책임의 판단이 필요한 부분은 실제 사실관계를 확보한 뒤 법률 전문가에게 확인한다.

6. 기여와 거버넌스

초기 기여는 DCO 1.1과 Signed-off-by로 받는다. DCO는 제출 권리와 출처에 대한 기여자의 진술이며 저작권 양도가 아니다. 공개된 sign-off의 개인정보가 기여 이력에 남을 수 있음을 알린다. DCO 1.1

Apache-2.0을 선택했다고 CLA가 자동으로 필수인 것은 아니다. 기업 기여·기존 코드 기증·향후 재라이선싱처럼 별도 약정이 필요한 때 검토한다. ASF 내부의 CLA 절차를 이 프로젝트의 의무로 복사하지 않는다. ASF Contributor Agreements

초기 관리자는 저장소 소유자가 지정한다. 실제로 없는 검토자·보안팀·지원 인력을 기재하지 않는다. 1인 단계의 자기 검토를 독립 검토로 표시하지 않고, 인원이 확보되면 권한·배포·서명 경계 변경에 별도 검토자를 지정한다. 세부 절차는 함께 제공한 기여·거버넌스 문서에 따른다.

7. GitHub 적용 순서

1. 별도 저장소에 허용한 최종 자료만 복사한다. 원본·임시 파일·개인 경로·회사 과제 파일·실제 운영 증적을 제외한다.
2. F_GitHub_공개준비/F_배포파일명_매핑.md대로 LICENSE·NOTICE·SECURITY·CONTRIBUTING·GOVERNANCE를 표준 이름으로 배치한다. 전달물의 F_ 접두어와 실제 GitHub 인식 이름을 구분한다.
3. 실제 권리자와 파일 범위를 확인하고, 파일별 SPDX와 필요한 제3자 고지를 정리한다. F_NOTICE.txt의 일반 기여자 표기를 근거 없는 실명·회사 소유권 주장으로 바꾸지 않는다.
4. 기본 브랜치 보호·최소 토큰 권한·비밀 검사·필요한 검수 작업을 설정한다. 코드가 아직 없으면 존재하지 않는 빌드 성공 배지를 넣지 않는다.
5. 비공개 취약점 제보 기능을 활성화하고 수신 경로를 점검한다. SECURITY 파일만 넣어서는 기능이 활성화되지 않는다. GitHub 설정 안내
6. README에 설계 단계, 구현 범위, 지원·미지원 기능, 검증 범위, 라이선스 경계를 공개한다. 제품 출시 시에는 이 상태를 실제 테스트 결과에 맞춰 갱신한다.
7. 코드·이미지를 배포하기 전 실제 포함물의 SBOM·고지·대응 소스·provenance·서명·체크섬을 함께 검수한다.

GitHub 공개 저장소라는 사실만으로 재사용 조건이 정해지는 것은 아니다. 루트의 표준 LICENSE와 명확한 범위 설명을 제공한다. GitHub: Licensing a repository

8. 공개 중단 조건과 남은 확인

다음이 발견되면 해당 파일·배포물을 공개 후보에서 제외하고 해결한다: 권리·출처 불명, 무단 회사 자료·개인정보·비밀 포함, Apache-only와 GPLv2-only의 미해결 직접 결합, eBPF source/license 불일치, 외부 모델·규칙·SDK의 무단 번들, 필요한 소스·고지 제공 불가.

현재 확인된 것은 공식 라이선스의 일반 구조와 이 설계의 공개 정책이다. **실제 소스 저장소의 전체 의존성, 배포 바이너리·이미지, 회사 계약, 이름·로고의 권리, 개별 관할의 면책 효력은 아직 검증된 사실이 아니다.** 이 자료를 배포 적합성 인증이나 법률 의견서로 표시하지 않는다.